

LAPORAN TUGAS BESAR

Sistem Inventarisasi Laboratorium



Kelompok 6

Anggota Kelompok :

Vanadia Valianti	13224011
Kenneth Harrison Siswanto	13224032
Beatrice Corryn Pangestu	13224038
Muhammad Aqeel Ghani	13224071

PEMECAHAN MASALAH DENGAN PEMROGRAMAN
EL2008

PROGRAM STUDI SARJANA TEKNIK ELEKTRO
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Daftar Isi.....	1
Daftar Tabel.....	4
Daftar Gambar.....	5
Bab 1: Pendahuluan.....	6
1.1. Latar Belakang Masalah.....	6
1.2. Tujuan Tugas Besar.....	6
1.3. Lingkup Sistem.....	6
Bab 2: Dekomposisi dan Solusi Masalah.....	8
2.1. Dekomposisi Masalah.....	8
2.1.1. Analisis Keterbatasan Perangkat Keras.....	8
2.1.2. Analisis Kebutuhan Sistem.....	9
2.2. Pendekatan Solusi.....	11
2.2.1. Pemilihan dan Perancangan Struktur Data.....	11
2.2.2. Perancangan Fungsi Sistem.....	15
Bab 3: Perancangan Sistem.....	22
3.1. Data Flow Diagram Sistem.....	22
3.1.1. Data Flow Diagram Level 0.....	22
3.1.2. Data Flow Diagram Level 1.....	22
3.2. Flowchart Sistem dan Subsystem.....	23
3.2.1. Flowchart FindCharIndex.....	23
3.2.2. Flowchart FindChar.....	24
3.2.3. Flowchart ConvertStringToByteArray.....	24
3.2.4. Flowchart ConvertByteArrayToString.....	25
3.2.5. Flowchart CompressDataNamaKomponen.....	26
3.2.6. Flowchart UnCompressDataNamaKomponen.....	27
3.2.7. Flowchart UpperString.....	28
3.2.8. Flowchart CompressDataPIC.....	28
3.2.9. Flowchart UnCompressDataPIC.....	29
3.2.10. Flowchart GetNamaKomponen.....	30
3.2.11. Flowchart GetLokasi.....	30
3.2.12. Flowchart GetStatusTersedia.....	31
3.2.13. Flowchart GetStatusDipinjam.....	31
3.2.14. Flowchart GetStatusRusak.....	31
3.2.15. Flowchart GetJumlahStock.....	32
3.2.16. Flowchart GetKategori.....	33
3.2.17. Flowchart GetPIC.....	33
3.2.18. Flowchart IsSlotEmpty.....	34
3.2.19. Flowchart GetEmptySlotCount.....	34
3.2.20. Flowchart GetEmptyId.....	35
3.2.21. Flowchart UpdateNamaKomponen.....	36

3.2.22. Flowchart UpdateLokasi.....	37
3.2.23. Flowchart UpdateStatusTersedia.....	38
3.2.24. Flowchart UpdateStatusDipinjam.....	38
3.2.25. Flowchart UpdateStatusRusak.....	39
3.2.26. Flowchart UpdateKategori.....	39
3.2.27. Flowchart UpdatePIC.....	40
3.2.28. Flowchart DeleteCatalogItem.....	40
3.2.29. Flowchart InitializeCatalog.....	41
3.2.30. Flowchart ReadEEPROM.....	41
3.2.31. Flowchart SaveEEPROM.....	42
3.2.32. Flowchart Fungsi Search By ID.....	43
3.2.33. Flowchart Fungsi Ringkasan.....	44
3.2.34. Flowchart Fungsi delete.....	45
3.2.35. Flowchart Fungsi updateStatus.....	46
3.2.36. Flowchart Fungsi showAllItem.....	47
3.2.37. Flowchart Fungsi showMenu.....	48
3.2.38. Flowchart Fungsi BacaInputAngka.....	48
3.2.39. Flowchart Fungsi runSystem.....	49
3.2.40. Flowchart Fungsi deleteAll.....	50
3.2.41. Flowchart Fungsi addData.....	51
3.2.42. Flowchart Fungsi updateStock.....	52
3.3. Penjelasan Logika dan Struktur Data.....	53
3.3.1. Penjelasan Struktur Data.....	53
3.3.2. Penjelasan Logika CRUD dan EEPROM.....	53
3.3.3. Penanganan Masalah Khusus.....	54
Bab 4: Implementasi dan Studi Kasus.....	56
4.1. Struktur Program Modular.....	56
4.1.1. main.cpp.....	56
4.1.2. constant.h dan constant.cpp.....	56
4.1.3. catalog.h dan catalog.cpp.....	56
4.1.4. eepromControl.h dan eepromControl.cpp.....	56
4.1.5. addData.h dan addData.cpp.....	56
4.1.6. delete.h dan delete.cpp.....	56
4.1.7. deleteAll.h dan deleteAll.cpp.....	56
4.1.8. Ringkasan.h dan Ringkasan.cpp.....	56
4.1.9. SearchByID.h dan SearchByID.cpp.....	56
4.1.10. serialUI.h dan serialUI.cpp.....	57
4.1.11. showAllItem.h dan showAllItem.cpp.....	57
4.1.12. updateStatus.h dan updateStatus.cpp.....	57
4.1.13. updateStock.h dan updateStock.cpp.....	57
4.2. Skenario Studi Kasus.....	57

4.2.1. Main Menu.....	57
4.2.2. Tambah Data Barang.....	57
4.2.3. Hapus Data Barang.....	59
4.2.4. Cari Data Berdasarkan ID.....	60
4.2.5. Perbarui Stock Barang.....	61
4.2.6. Perbarui Status.....	61
4.2.7. Tampilkan Semua Item.....	62
4.2.8. Ringkasan Inventaris.....	63
4.2.9. Save to EEPROM.....	63
Bab 5: Analisis Hasil dan Evaluasi.....	65
5.1. Analisis Implementasi.....	65
5.2. Analisis Keterbatasan Sistem.....	65
Bab 6: Pembagian Tugas Anggota Kelompok.....	66
Bab 7: Penutup.....	67
7.1. Kesimpulan.....	67
7.2. Saran Pengembangan.....	67
Daftar Pustaka.....	68
Lampiran.....	69

Daftar Tabel

<i>Tabel 2.1.2.1 Kebutuhan Data Inventarisasi.....</i>	<i>9</i>
<i>Tabel 2.2.1.1 Dictionary Karakter Terkompresi.....</i>	<i>12</i>
<i>Tabel 2.2.1.2 Indeks Kategori.....</i>	<i>13</i>
<i>Tabel 2.2.1.3 Rincian Besar Struct Struktur Data.....</i>	<i>14</i>
<i>Tabel 3.3.1.1 Rancangan Struct dalam Kode Program.....</i>	<i>53</i>

Daftar Gambar

<i>Gambar 2.1.1.1 AVR Memory Map (Architecture Used by Arduino UNO) [2].....</i>	<i>8</i>
<i>Gambar 2.2.1.1 Ilustrasi Compacting.....</i>	<i>14</i>
<i>Gambar 2.2.1.2 Ilustrasi EEPROM.....</i>	<i>15</i>
<i>Gambar 3.3.2.1 Ilustrasi Penggabungan Lokasi dan Jumlah Status.....</i>	<i>54</i>
<i>Gambar 5.1.1 Analisis Penggunaan Memori Program.....</i>	<i>65</i>

Bab 1: Pendahuluan

1.1. Latar Belakang Masalah

Laboratorium Embedded Systems memiliki banyak komponen dan peralatan yang digunakan untuk praktikum, penelitian, dan proyek mahasiswa, seperti sensor, Arduino, ESP32, Raspberry Pi, modul komunikasi, motor, driver motor, multimeter, kabel, dan perangkat pendukung lainnya. Selama ini, inventarisasi barang masih dilakukan secara manual menggunakan logbook atau spreadsheet sederhana. Akibatnya, data sering tidak sesuai dengan kondisi aktual, pencarian alat membutuhkan waktu lama, dan history peminjaman atau pengembalian tidak selalu tercatat dengan baik.

Untuk mendukung digitalisasi laboratorium, sebuah sistem inventarisasi sederhana akan dibuat berbasis Arduino Uno. Setiap komponen/peralatan laboratorium diberi ID unik yang akan menjadi kata kunci untuk mengakses data inventaris. Sistem ini tidak akan menggunakan sistem database yang sudah ada karena Arduino Uno memiliki keterbatasan sumber daya, khususnya memori yang kecil.

1.2. Tujuan Tugas Besar

Tujuan dengan dilakukannya tugas besar ini adalah :

1. Mendekomposisi permasalahan serta menyusun solusi yang mempertimbangkan kebutuhan pengguna dan keterbatasan Arduino Uno.
2. Merancang sistem secara modular dan terdiri dari berbagai subsistem yang saling terintegrasi.
3. Mengimplementasikan sistem berbasis Arduino Uno yang menggunakan bahasa pemrograman C/C++.
4. Menganalisis hasil serta mengevaluasi keterbatasan sistem yang telah dirancang dan diimplementasikan.
5. Melakukan pembagian tugas antar anggota kelompok guna memberi kesempatan tiap anggota kelompok untuk berkontribusi secara substansial.

1.3. Lingkup Sistem

Dalam rangka menjaga fokus tugas besar ini untuk memenuhi capaian pembelajaran Mata Kuliah Pemecahan Masalah dengan Pemrograman (EL2008), berikut merupakan lingkup sistem yang akan dirancang :

1. Tugas besar ini berfokus pada perancangan sistem *database* pada *microcontroller* Arduino Uno.
2. Dalam mengimplementasikan sistem pada Arduino Uno, akan digunakan *framework* Arduino yang berbasis C++ sehingga dapat digunakannya *library* Arduino seperti Serial yang mengurus komunikasi UART antara Arduino Uno dengan komputer, dan EEPROM yang memungkinkan dilakukannya penyimpanan pada memori *non-volatile* sehingga Arduino Uno dapat menyimpan *database* walau dalam keadaan tidak dinyalakan.
3. Penulisan program akan menggunakan file “.h” (*Header File*) untuk menuliskan deklarasi fungsi, prosedur, konstanta, dan struktur data program, serta file “.cpp” (*Source File*) untuk menuliskan implementasi program.

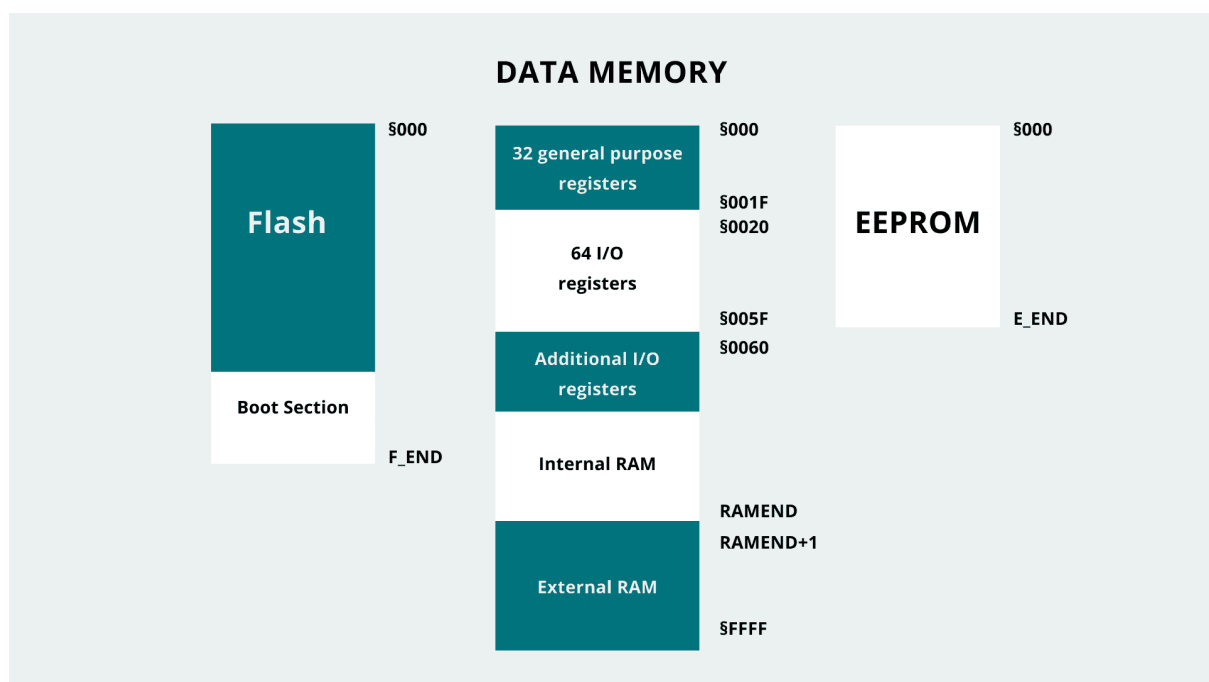
4. Penggunaan C++ dalam tugas besar ini hanya sebatas untuk dapat menggunakan *library* Arduino sehingga tidak perlunya dilakukan penulisan program atau *driver* untuk memungkinkan Arduino melakukan komunikasi serial dan penyimpanan dalam EEPROM. Hal ini dikarenakan hal tersebut sudah diluar capaian Mata Kuliah EL2008.
5. Selain penggunaan *library* Arduino, program akan ditulis dalam bahasa pemrograman C yang masih dapat ditulis dalam file “.cpp” dikarenakan C++ kurang lebih merupakan *superset* dari C.
6. Sistem akan menggunakan *Serial Monitor* sebagai *interface* antar sistem dengan pengguna.

Bab 2: Dekomposisi dan Solusi Masalah

2.1. Dekomposisi Masalah

2.1.1. Analisis Keterbatasan Perangkat Keras

Arduino UNO R3 merupakan *microcontroller board* yang menggunakan *chip* ATmega328P. ATmega328p memiliki spesifikasi *clock speed* 16 MHz, 2 KB SRAM, 32 KB Flash, dan 1 KB EEPROM [1]. RAM (*Random-Access Memory*) dalam sistem mikrokontroler adalah sistem penyimpanan sementara data atau *run-time data*. Pada Arduino UNO R3, jenis RAM yang digunakan adalah SRAM (*Static Random-Access Memory*) yaitu RAM yang menggunakan *flip-flop* untuk menyimpan satu bit data. SRAM dalam Arduino UNO R3 dibagi menjadi beberapa seksi : Text, Data, BSS, Stack, dan Heap. Memori Flash merupakan bagian dari ROM (*Read-Only Memory*) suatu mikrokontroler seperti Arduino UNO R3. Flash digunakan dalam menyimpan *firmware* sistem yang akan dijalankan oleh Arduino UNO. EEPROM (*Erasable Programmable Read-Only Memory*) adalah memori dalam Arduino UNO yang dapat ditulis dan dihapus serta bersifat *non-volatile* (dapat mempertahankan data walaupun tidak diberikan daya). EEPROM memiliki batas penggunaan penulisan ulang sebanyak 100.000 siklus [2].



Gambar 2.1.1.1 AVR Memory Map (Architecture Used by Arduino UNO) [2]

Dalam merancang sistem inventarisasi, banyak memori yang dimiliki oleh perangkat keras akan mempengaruhi bagaimana sistem ini akan dibuat. Dikarenakan dalam suatu sistem inventaris dibutuhkan supaya data yang tersimpan tidak hilang saat Arduino UNO tidak diberi daya, maka memori yang menjadi perhatian utama sistem ini dan menjadi *bottleneck* adalah

EEPROM. *Chip* ATmega328P memiliki EEPROM sebesar 1 KB atau 1024 Byte.

2.1.2. Analisis Kebutuhan Sistem

Sistem inventarisasi memiliki kebutuhan data per barang sebagai berikut :

Tabel 2.1.2.1 Kebutuhan Data Inventarisasi

Data	Deskripsi
ID Barang	ID yang diberikan kepada tiap tipe barang.
Nama Barang	Nama yang dimiliki oleh barang (terdiri dari 8 karakter antara huruf atau angka)
Lokasi Barang	Lokasi penyimpanan barang. Lokasi diasumsikan berbentuk bagaikan suatu loker 8×8 dengan nomor loker 0-63.
Jumlah Barang (dan Status) : • Jumlah Status Tersedia • Jumlah Status Dipinjam • Jumlah Status Rusak	Jumlah barang yang dibagi menjadi per status barang (tersedia, dipinjam, dan rusak). Untuk total barang yang terdata dapat dijumlah ketiga jumlah status barang tersebut.
Kategori Barang	Kategori untuk barang. Kategori barang terdiri dari : • Mikon • SBC • Sensor • Aktuator • Comm • Display • Pasif • Semikon • IC • Power • Driver • AlatUkur • Instrumen • Alat • Media • Connector
PIC/Pemilik	PIC dari suatu barang. PIC akan disimpan sebagai abreviasi nama mereka (terdiri dari 4 karakter). PIC dan Pemilik diasumsikan sama dikarenakan jika dipisah, kemungkinan besar pemilik barang tersebut adalah Laboratorium Embedded Systems.

Lalu, berikut adalah kebutuhan fungsi atau fitur sistem inventarisasi :

1. Search by ID

Fungsi yang mencari data inventaris berdasarkan input ID dari user. Beberapa hal yang harus diperhatikan pada implementasi fungsi ini adalah pencarian dilakukan tanpa looping agar tidak membebani prosesor. Selain itu, harus cek *Out-of-Bound* dan *NULL* memory yang memastikan ID yang dimasukkan oleh User ada dalam catalog.

2. Ringkasan Inventory

Fungsi ringkasan inventory dibuat untuk memberikan ringkasan penting dari sistem yang ada. Mulai dari slot memori yang tersisa di EEPROM, sampai ringkasan total barang yang ada (siap digunakan, sedang dipinjam, ataupun kondisi rusak).

3. Delete Item

Fungsi yang digunakan untuk menghapus data barang pada sistem inventaris dengan cara mengosongkan semua atribut di dalam struktur data, sehingga data tersebut tidak lagi aktif dan slotnya bisa digunakan kembali

4. Update Status

Fungsi untuk memperbarui jumlah barang berdasarkan status tertentu pada sistem inventaris, seperti tersedia, dipinjam, dan rusak, sesuai dengan nilai baru yang diberikan pada data barang.

5. Show All Item

Fungsi ini digunakan untuk menampilkan seluruh data barang yang tersimpan dalam sistem inventaris secara berurutan berdasarkan indeks array (0 hingga 63). Sistem akan melewati slot yang kosong dan hanya menampilkan data yang masih aktif, sehingga informasi yang ditampilkan lebih terstruktur dan mudah dibaca oleh pengguna.

6. Add Item

Fungsi yang digunakan untuk menambahkan data barang baru ke dalam sistem inventaris. Fungsi ini secara otomatis mencari slot ID kosong yang tersedia dan melakukan pengecekan kapasitas memori di awal untuk memastikan masih ada ruang penyimpanan. Selain itu, fungsi ini dilengkapi dengan sistem validasi input untuk memastikan data yang dimasukkan oleh pengguna seperti indeks kategori (0-15), lokasi (0-63), dan jumlah stok (tersedia, dipinjam, rusak) selalu berada pada batas yang diizinkan (tidak *overflow* atau negatif) sebelum menyimpannya ke dalam struktur data.

7. Update Stock

Fungsi yang digunakan untuk menyesuaikan jumlah stok barang yang tersedia di dalam sistem inventaris secara dinamis. Pengguna dapat langsung menambah atau mengurangi jumlah stok yang ada menggunakan input nilai positif maupun negatif.

8. Delete All

Fungsi yang digunakan untuk menghapus semua list item yang ada di penyimpanan SRAM maupun EEPROM (sesuai pilihan dari user). Pengguna diminta konfirmasi penghapusan setelah memilih menu ini.

9. Serial UI

Fungsi sistem utama dari keseluruhan program yang dibuat. Fungsi ini memberikan tampilan menu, melakukan parsing input pengguna, dan juga menjalankan semua fungsi yang ada sesuai dengan pilihan dari pengguna.

2.2. Pendekatan Solusi

2.2.1. Pemilihan dan Perancangan Struktur Data

Dikarenakan keterbatasan memori Arduino UNO yang digunakan, penggunaan Arduino UNO yang dikhususkan dalam menjalankan sistem inventarisasi, dan batas atas banyak barang yang dapat ditentukan diawal, maka struktur data yang akan diimplementasikan adalah **array**. Berikut adalah alasan pemilihan array :

- Kemudahan dalam membaca dan menulis EEPROM. EEPROM dalam Arduino UNO dibaca bagaikan suatu array dengan indeks per byte nya. Maka dengan menggunakan array, EEPROM dapat menerima data dari SRAM tanpa harus melakukan konversi terlebih dahulu dan juga sebaliknya.
- Mudah dalam menentukan banyak slot kosong yang masih tersisa. Dikarenakan jumlah maksimum slot yang bisa diberikan itu statik dan berdasarkan perhitungan sebelum implementasi, untuk menghitung slot kosong dapat dilakukan dengan mengurangi jumlah slot total lalu dikurangi slot terpakai, atau dengan menghitung slot kosong yang ada dalam array (slot dengan mark kosong).
- Tidak perlunya menyiapkan *overhead* untuk *pointer* dan *address* seperti linked list. Pointer dalam sistem 8-bit seperti Arduino UNO memerlukan ruang sebesar 2 byte yang bisa saja digunakan untuk data lain.
- Tidak perlunya menuliskan ID secara eksplisit. Dikarenakan tiap slot array memiliki indeks sendiri, maka jika indeks tersebut digunakan sebagai ID, tidak perlunya dilakukan penyimpanan ID.
- Cepatnya dalam melakukan pencarian berdasarkan ID. Dikarenakan ID barang merupakan indeks dari slotnya, maka kecepatan dalam mencari barang tersebut adalah $O(1)$.

Setelah menentukan struktur data keseluruhan, perlu dilakukan penentuan rancangan struct beserta elemen-elemennya. Pertama, untuk ID barang dikarenakan digunakan array, maka ID barang tidak perlu dijadikan elemen struct dikarenakan dapat dilihat dari indeksnya atau posisinya di array tersebut.

Untuk data nama barang dan PIC, kedua data tersebut akan bertipe string. Namun, dikarenakan tidak semua karakter ASCII akan digunakan dalam menuliskan kedua data tersebut, dapat dilakukan kompresi dengan hanya menggunakan karakter yang perlu saja. Pertama, ada dua karakter yang perlu ada yaitu null (direpresentasikan sebagai '+') untuk menandakan akhir string atau string kosong dan space (direpresentasikan sebagai '-') jika diperlukan dalam memisahkan dua kata. Untuk PIC dikarenakan hanya akan berupa abreviasi, maka karakter yang akan digunakan hanya huruf yang tidak *case sensitive*. Maka total karakter yang dibutuhkan adalah $26+2 = 28$ untuk PIC tetapi dikarenakan komputer menggunakan sistem base 2 maka jumlah bit yang paling mendekati adalah 5 bit = 32 karakter. Untuk nama barang akan digunakan huruf dan angka yang jika ditotal berjumlah $26+10+2 = 38$. Namun, dikarenakan jumlah bit yang paling mendekati adalah 6 bit = 64 karakter, masih tersisa 26 slot karakter yang masih bisa digunakan. Slot ini cukup besar untuk dapat membuat hurufnya *case sensitive* (terdiri dari huruf kapital dan non-kapital). Maka total karakter yang digunakan akan berjumlah $2+10+26*2 = 64$. Berikut adalah tabel *dictionary* antara indeks dengan karakter untuk sistem diatas.

Tabel 2.2.1.1 Dictionary Karakter Terkompresi

6 Bit															
5 Bit								Karakter yang tidak digunakan 5 Bit							
0	+	8	G	16	O	24	W	32	e	40	m	48	u	56	2
1	-	9	H	17	P	25	X	33	f	41	n	49	v	57	3
2	A	10	I	18	Q	26	Y	34	g	42	o	50	w	58	4
3	B	11	J	19	R	27	Z	35	h	43	p	51	x	59	5
4	C	12	K	20	S	28	a	36	i	44	q	52	y	60	6
5	D	13	L	21	T	29	b	37	j	45	r	53	z	61	7
6	E	14	M	22	U	30	c	38	k	46	s	54	0	62	8
7	F	15	N	23	V	31	d	39	l	47	t	55	1	63	9

Dikarenakan nama komponen terdiri dari 8 karakter maka total ruang digunakan adalah $6*8 = 48$ bit. Sedangkan untuk PIC dikarenakan terdiri dari 4 karakter maka total ruang yang digunakan adalah $5*4 = 20$ bit.

Selanjutnya adalah lokasi barang yang diasumsikan sebagai loker 8×8 . Dikarenakan loker 8×8 itu sama dengan 64 loker, maka dapat direpresentasikan sebagai 6 bit.

Lalu untuk kategori dikarenakan terdapat total 16 kategori maka data tersebut direpresentasikan sebagai indeks 4 bit.

Tabel 2.2.1.2 Indeks Kategori

Indeks	Kategori
0	Mikon
1	SBC
2	Sensor
3	Aktuator
4	Comm
5	Display
6	Pasif
7	Semikon
8	IC
9	Driver
10	Power
11	AlatUkur
12	Instrumen
13	Alat
14	Media
15	Connector

Untuk jumlah barang beserta jumlah status, dikarenakan sebenarnya batasannya bisa tak terhingga, maka harus diambil asumsi berapa maksimum jumlah yang dapat ada. Dikarenakan rancangan struktur data sebelumnya sudah terdapat $48+20+6+4 = 78$ bit dan supaya total bit per struct merupakan kelipatan 8 (satu byte) maka dapat diambil 6 bit per jumlah status (maksimal 63 *item* per status) yang menghasilkan jumlah bit untuk jumlah total adalah $6*3 = 18$ bit (maksimal $63*3 = 189$ *item* total dengan kondisi setiap status terdiri dari 63 *item*).

Oleh karena itu, besar satu struct untuk struktur data ini adalah $48+20+6+4+18 = 96$ bit = 12 byte. Namun, dikarenakan dalam c suatu variabel hanya bisa

sekecil satu byte, maka dapat dilakukan *compacting* dengan menggabungkan variabel-variabel ini sehingga dapat disimpan tanpa adanya padding.

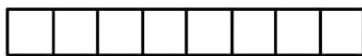
Tabel 2.2.1.3 Rincian Besar Struct Struktur Data

Data	Bit	Byte
ID Barang	0	0
Nama Komponen	$6 \times 8 = 48$	6
Lokasi	6	3
Jumlah dan Status	$6 \times 3 = 18$	
Kategori	4	3
PIC	$5 \times 4 = 20$	
Total Byte		12

Legenda



: Bit

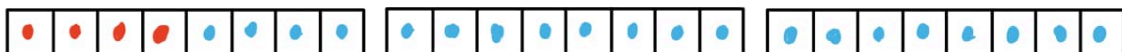


: Byte

Ilustrasi: Lokasi dengan Jumlah dan Status
 ● Lokasi ● Jumlah dan Status



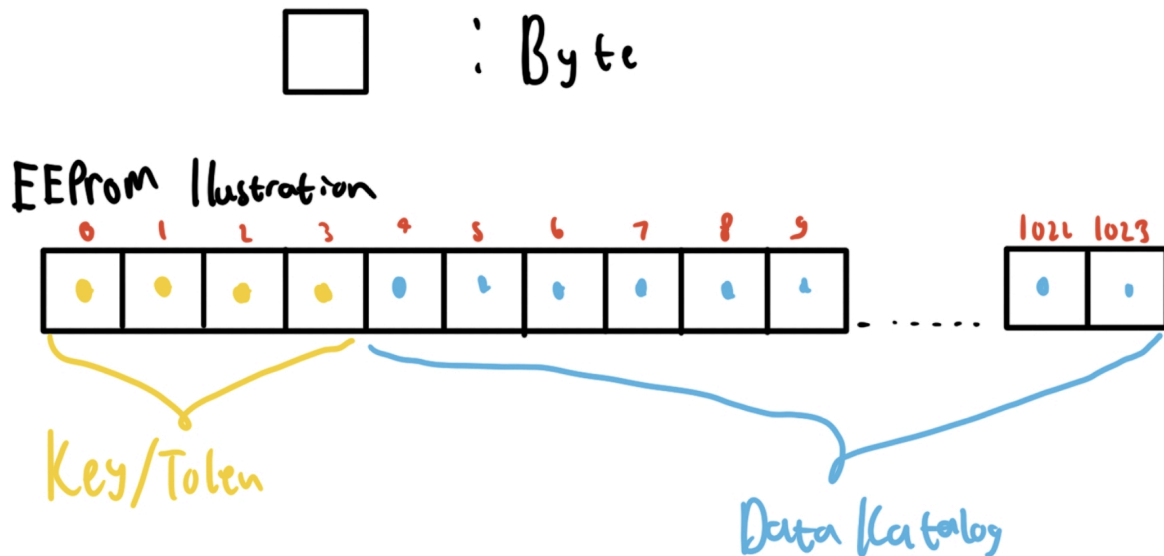
Ilustrasi: Kategori dengan PIC
 ● Kategori ● PIC



Gambar 2.2.1.1 Ilustrasi Compacting

Dikarenakan sudah diketahui bahwa setiap struct memiliki ruang 12 byte, jumlah total barang yang dapat disimpan dalam sistem inventaris dapat dihitung dengan membagi ruang di EEPROM dengan ruang satu struct barang. Oleh karena itu didapatkan banyak barang yang bisa disimpan sebesar $1024 // 12 = 85$ barang. Namun, masih terdapat ruang tersisa di EEPROM sebesar $1024 \% 12 = 4$ byte. Hal ini memberikan oportunitas dan ruang untuk

mengimplementasikan key atau token di 4 byte awal EEPROM sebagai penanda apakah isi EEPROM bisa dipakai sebagai data atau dianggap kosong saja.



Gambar 2.2.1.2 Ilustrasi EEPROM

2.2.2. Perancangan Fungsi Sistem

Dikarenakan menggunakan struktur data dengan struct yang tidak biasa, maka diperlukannya fungsi CRUD (*Create, Read, Update, Delete*) untuk mempermudah dalam membuat fungsi yang lebih kompleks selanjutnya.

Pertama, ini merupakan fungsi-fungsi penopang CRUD terutama untuk data yang menggunakan string yang akan di-*compress* dan di-*uncompress* :

- FindCharIndex
Fungsi mencari index karakter (berdasarkan tabel 2.2.1.1)
- FindChar
Fungsi mencari character (berdasarkan tabel 2.2.1.1)
- ConvertStringToByteArray
Prosedur mengubah string menjadi byte array
- ConvertByteArrayToString
Prosedur mengubah byte array menjadi string
- CompressDataNamaKomponen
Prosedur mengompres nama komponen
- UnCompressDataNamaKomponen
Prosedur mengkompres nama komponen

- `UpperString`
Prosedur membuat kapital seluruh karakter string
- `CompressDataPIC`
Prosedur mengompres data PIC
- `UnCompressDataPIC`
Prosedur menuncompress data PIC

Berikut merupakan fungsi *Get/Read* untuk membaca struktur data:

- `GetNamaKomponen`
Prosedur Mendapat Nama Komponen
- `GetLokasi`
Fungsi mendapatkan lokasi item
- `GetStatusTersedia`
Fungsi mendapatkan jumlah item dengan status tersedia
- `GetStatusDipinjam`
Fungsi mendapatkan jumlah item dengan status dipinjam
- `GetStatusRusak`
Fungsi mendapatkan jumlah item dengan status rusak
- `GetJumlahStock`
Fungsi mendapatkan jumlah stock item
- `GetKategori`
Fungsi mendapatkan kategori item
- `GetPIC`
Prosedur Mendapat PIC
- `IsSlotEmpty`
Fungsi Menentukan Apakah Slot dalam Katalog Kosong
- `GetEmptySlotCount`
Fungsi Menghitung Slot Kosong
- `GetEmptyId`
Fungsi Mencari Id Slot Kosong

Berikut merupakan fungsi *Update* untuk mengubah struktur data:

- `UpdateNamaKomponen`
Prosedur Mengubah Nama Komponen
- `UpdateLokasi`
Prosedur mengupdate lokasi item

- UpdateStatusTersedia
Prosedur mengupdate jumlah item dengan status tersedia
- UpdateStatusDipinjam
Prosedur mengupdate jumlah item dengan status dipinjam
- UpdateStatusRusak
Prosedur mengupdate jumlah item dengan status rusak
- UpdateKategori
Prosedur mengupdate kategori Item
- UpdatePIC
Prosedur Mengubah PIC

Berikut merupakan fungsi *Delete* untuk menghapus struktur data:

- DeleteCatalogItem
Prosedur Mendelete Item
- InitializeCatalog
Prosedur Inisialisasi Katalog atau delete all item

Berikut merupakan fungsi yang bertujuan untuk melakukan pembacaan dan penyimpanan di EEPROM:

- ReadEEPROM
Prosedur membaca data dari EEPROM jika key sesuai
- SaveEEPROM
Prosedur menyimpan data ke EEPROM

1. Search by ID

Solusi yang didapatkan dari dekomposisi masalah antara lain:

- Pencarian dengan konstanta waktu $O(1)$, dengan memanfaatkan ID sebagai index array dari *catalog* itu sendiri.
- Memberikan validasi bahwa ID masih di dalam rentang, jika input ID di luar *size* dari EEPROM itu sendiri.
- Memberikan validasi bahwa ID tersebut kosong atau tidak dengan mengecek 1 byte awal pada nama, jika nama sudah *NULL* maka langsung saja tidak perlu mengecek satu blok memori.
- Menggunakan buffer 40 byte SRAM (memori sementara) hanya untuk output *text* dan akan langsung dikosongkan.
- Menggunakan fungsi *low-level* dekompresi agar otomatis membuka *compressed* byte menjadi char dan int.

Keterbatasan:

- Input ID harus benar angka tidak negatif

- Perlu identifikasi besar memori/array katalog dari main
- User harus tahu ID barang, bukan nama barang

2. Ringkasan Inventory

Solusi yang didapatkan dari dekomposisi masalah antara lain:

- Melakukan loop notasi $O(n)$ dengan n sebagai banyaknya index di array katalog digunakan untuk mengecek dari awal array katalog sampai selesai dan menjumlah variabel slot available dan status dari ID yang tidak kosong.
- Melakukan perhitungan sisa slot dari size array full dikurang dengan slot terpakai.
- Memberikan notifikasi jika slot yang tersedia hanya tersisa 5 atau kurang.
- Melakukan perhitungan grand total dengan menjumlahkan semua stok barang dari ketiga status yang ada.
- Melakukan pencetakan output dalam bentuk tabel yang rapi.

Keterbatasan:

- Tidak dapat menampilkan history action yang terjadi pada inventory karena keterbatasan Arduino.
- Notasi $O(n)$ yang lumayan memakan waktu jika inventarisasi dikembangkan dengan memori sangat besar.
- Notifikasi tidak dapat memberikan ID spesifik untuk slot yang kosong.

3. Delete Item

Solusi yang didapatkan dari dekomposisi masalah antara lain:

- Menghapus data barang pada sistem inventaris dengan cara mengosongkan seluruh atribut yang terdapat pada struktur data CatalogItem.
- Mengatur ulang nilai setiap field seperti nama komponen, lokasi, status, kategori, dan PIC menjadi nilai default sehingga data dianggap tidak aktif.
- Menggunakan pendekatan reset data pada level struktur tanpa melakukan penghapusan memori secara fisik.
- Memanfaatkan pengecekan sederhana pada atribut utama untuk menentukan apakah suatu slot data masih terisi atau sudah kosong.
- Slot yang telah direset dapat digunakan kembali untuk penyimpanan data baru tanpa perlu proses alokasi tambahan.

Keterbatasan:

- Data tidak benar-benar dihapus dari memori, hanya direset sehingga masih ada kemungkinan tertimpa data baru.
- Tidak ada fitur konfirmasi tingkat lanjut seperti autentikasi pengguna sebelum proses penghapusan dilakukan.
- Penghapusan hanya berbasis struktur data, belum terintegrasi dengan sistem log atau histori perubahan data.
- Tidak terdapat mekanisme recovery jika data terhapus secara tidak sengaja.

4. Update Status

Solusi yang diperoleh dari dekomposisi masalah antara lain:

- Melakukan pembaruan nilai status barang berdasarkan jenis status seperti tersedia, dipinjam, dan rusak sesuai input terbaru dari pengguna.
- Menggunakan teknik pengolahan data berbasis bit untuk menyimpan beberapa status dalam satu variabel agar lebih efisien dalam penggunaan memori.
- Melakukan validasi nilai input agar tidak melebihi kapasitas penyimpanan yang telah ditentukan oleh jumlah bit yang tersedia.
- Menggabungkan nilai baru dengan nilai lama menggunakan operasi bit *shifting* dan *masking* sehingga data lain dalam satu variabel tidak ikut berubah.
- Menyediakan fungsi pembacaan data untuk memastikan hasil pembaruan status dapat diverifikasi secara langsung setelah proses update dilakukan.

Keterbatasan:

- Nilai status dibatasi oleh kapasitas bit sehingga hanya mendukung rentang nilai tertentu (maksimal 63).
- Pengolahan data menggunakan bit manipulation membuat sistem kurang mudah dibaca dan dipahami tanpa dokumentasi tambahan.
- Tidak ada validasi input lanjutan seperti pengecekan konsistensi antar status (misalnya total stok tidak boleh negatif).
- Perubahan status tidak disertai riwayat perubahan sehingga tidak dapat dilacak versi sebelumnya.

5. Show All Item

Solusi yang diperoleh dari dekomposisi masalah antara lain:

- Melakukan iterasi data inventaris dari indeks 0 sampai batas maksimal 63 untuk memastikan seluruh slot diperiksa.
- Melakukan pengecekan apakah suatu slot kosong atau tidak menggunakan fungsi `IsSlotEmpty` agar data yang tidak aktif tidak ditampilkan.
- Mengambil data setiap item menggunakan fungsi getter seperti `GetNamaKomponen`, `GetPIC`, `GetLokasi`, `GetStatusTersedia`, `GetStatusDipinjam`, `GetStatusRusak`, dan `GetKategori`.
- Menampilkan data dalam format terstruktur melalui Serial Monitor agar informasi mudah dibaca oleh pengguna.
- Mengabaikan data kosong menggunakan mekanisme `continue` pada proses iterasi.
- Menggunakan buffer tampilan sementara untuk memastikan proses output berjalan stabil pada sistem embedded.

Keterbatasan:

- Sistem hanya menampilkan data berdasarkan indeks tetap (0–63) sehingga tidak mendukung pencarian dinamis.
- Tidak terdapat fitur sorting data berdasarkan kategori, nama, atau jumlah stok.
- Data yang ditampilkan bersifat statis dan tidak dapat difilter oleh pengguna.

6. Add Item

Solusi yang diperoleh dari dekomposisi masalah antara lain:

- Secara otomatis mencari slot ID kosong menggunakan fungsi iterasi mandiri sehingga mempermudah penempatan data di memori tanpa harus ditentukan secara manual oleh pengguna.
- Melakukan pengecekan sisa kapasitas memori di awal sebelum proses input dimulai untuk mencegah terjadinya kegagalan penulisan atau korupsi data pada EEPROM.
- Memiliki sistem validasi input berlapis yang mencegah nilai negatif atau angka di luar batas memori, sehingga memastikan data aman sebelum dikonversi dan disimpan ke dalam struktur data.
- Memanfaatkan sistem kompresi data string secara langsung saat proses input agar penggunaan ruang memori menjadi sangat efisien.

Keterbatasan:

- Pencarian slot kosong mengharuskan sistem melakukan iterasi (looping) array dari awal, yang berpotensi memakan waktu pemrosesan linear ($O(n)$) pada mikrokontroler jika jumlah data sudah mendekati kapasitas maksimal.
- Pengguna harus mengetahui referensi daftar indeks kategori (0-15) saat melakukan input, dikarenakan pembatasan memori yang tidak memungkinkan input berbasis teks secara bebas.
- Data yang masuk akan disimpan berdasarkan slot ID kosong pertama yang ditemukan, sehingga tidak ada fitur pengurutan (sorting) otomatis berdasarkan kategori atau nama.

7. Update Stock

Solusi yang diperoleh dari dekomposisi masalah antara lain:

- Mendukung penyesuaian stok secara dinamis dan fleksibel, di mana pengguna dapat langsung menambah atau mengurangi stok menggunakan nilai positif maupun negatif.
- Memiliki sistem proteksi logika dan validasi kalkulasi untuk mencegah jumlah stok menjadi negatif akibat pengurangan berlebih.
- Mencegah terjadinya masalah *overflow* pada rentang data bit dengan memblokir pembaruan jika hasil penjumlahan melewati batas maksimal kapasitas memori (63 buah).
- Modifikasi data menggunakan operasi manipulasi bit secara spesifik, sehingga pembaruan data stok dapat dilakukan secara efisien tanpa merusak data atribut lain di dalam satu memori yang sama.

Keterbatasan:

- Kapasitas maksimum stok dibatasi pada angka 63 item per status (tersedia, dipinjam, rusak) karena alokasi struktur data hanya menggunakan 6-bit per komponen.
- Perubahan jumlah stok tidak menyimpan riwayat histori (*log history*) penambahan atau pengurangan, sehingga administrator tidak dapat melacak kapan atau oleh siapa perubahan tersebut dilakukan.
- Pengguna harus mengetahui dan memasukkan ID barang secara spesifik untuk melakukan *update*, karena fungsi ini bergantung penuh pada pencarian berbasis array indeks, bukan berdasarkan teks atau nama komponen.

8. Delete All

Solusi yang diperoleh dari dekomposisi masalah antara lain:

- Melakukan loop `Serial.available() == 0` untuk menunggu input dari user dan melakukan loop `Serial.available() > 0` untuk parsing input sebagai char tanpa mengambil `\n` ataupun `\r`
- Menggunakan fungsi `InitializeCatalog()` untuk mereset ulang isi catalog menjadi kosong di SRAM.
- Menggunakan fungsi `saveEEPROM()` untuk menyimpan perubahan catalog SRAM yang menjadi kosong tersebut di EEPROM.

Keterbatasan:

- Tidak ada keterbatasan

9. Serial UI

Solusi yang diperoleh dari dekomposisi masalah antara lain:

- Membuat fungsi `showMenu()` secara terpisah agar fungsi rapi.
- Membuat fungsi parsing integer terpisah untuk input ID menggunakan loop `Serial.available() == 0` untuk menunggu input dan `Serial.parseInt()` untuk mengambil input ID nya sebagai integer.
- Melakukan loop `Serial.available() == 0` untuk menunggu input dari user dan melakukan loop `Serial.available() > 0` untuk parsing input menu sebagai char tanpa mengambil `\n` ataupun `\r`
- Menjalankan fungsi sesuai dengan pilihan menu dari pengguna, jika fungsi memerlukan input ID, digunakan fungsi parsing integer sebelumnya.

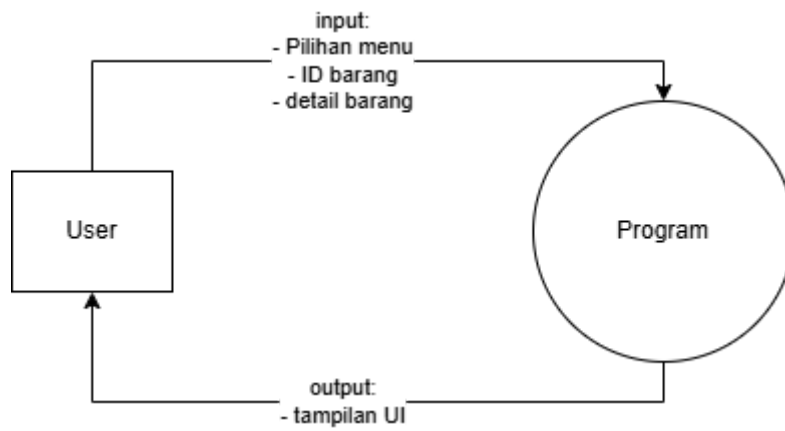
Keterbatasan:

- Sistem berhenti total saat menunggu input pengguna di Serial Monitor, sehingga Arduino tidak dapat menjalankan tugas latar belakang (seperti membaca sensor) secara bersamaan.
- Pembuangan sisa karakter antrean Serial membuat pengguna tidak bisa mengirim beberapa perintah sekaligus dalam satu baris secara beruntun.
- Fungsi `delay(1000)` membekukan sistem selama 1 detik, sehingga berisiko kehilangan data penting jika ada perangkat luar yang mengirimkan sinyal tepat pada durasi tersebut.

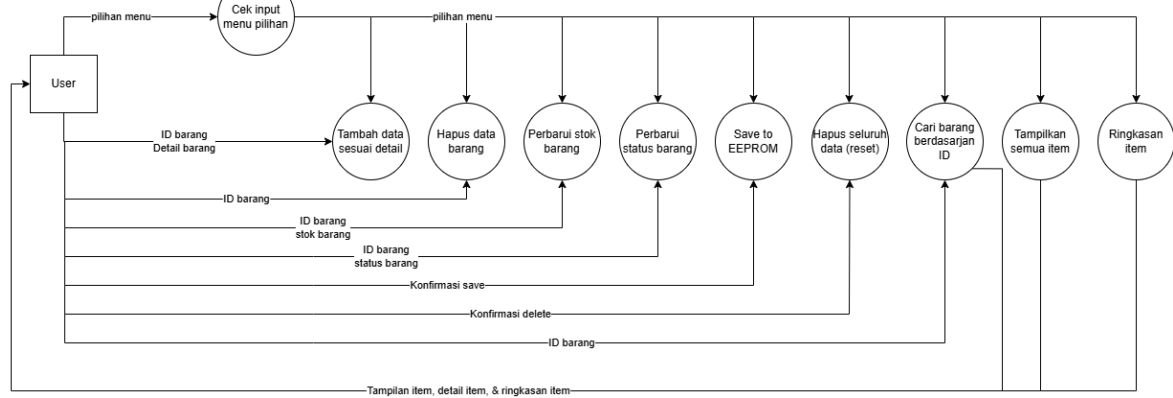
Bab 3: Perancangan Sistem

3.1. Data Flow Diagram Sistem

3.1.1. Data Flow Diagram Level 0

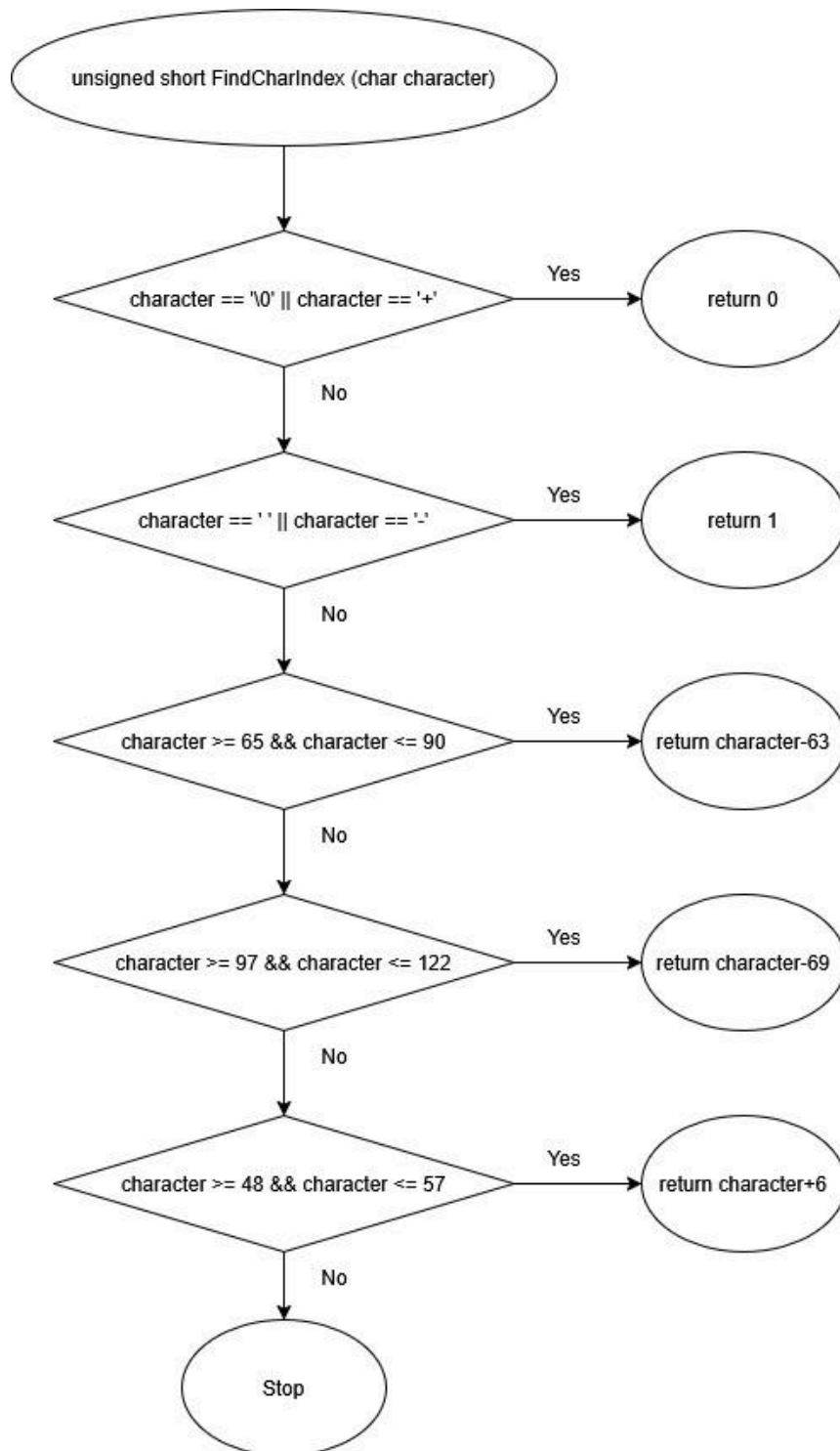


3.1.2. Data Flow Diagram Level 1

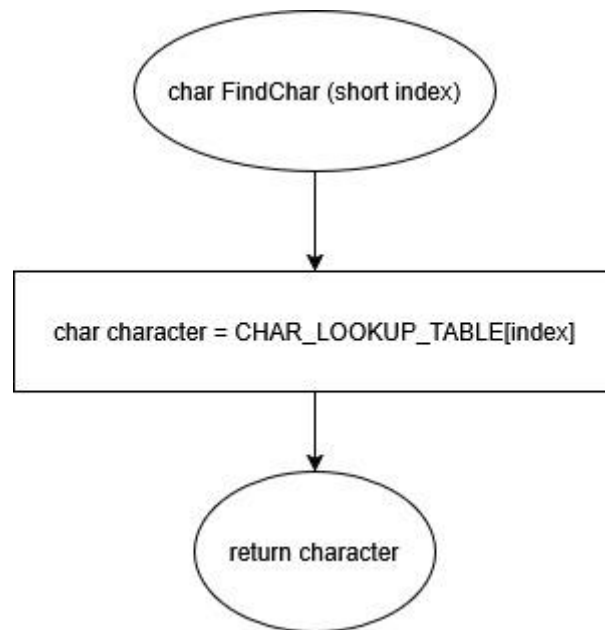


3.2. Flowchart Sistem dan Subsystem

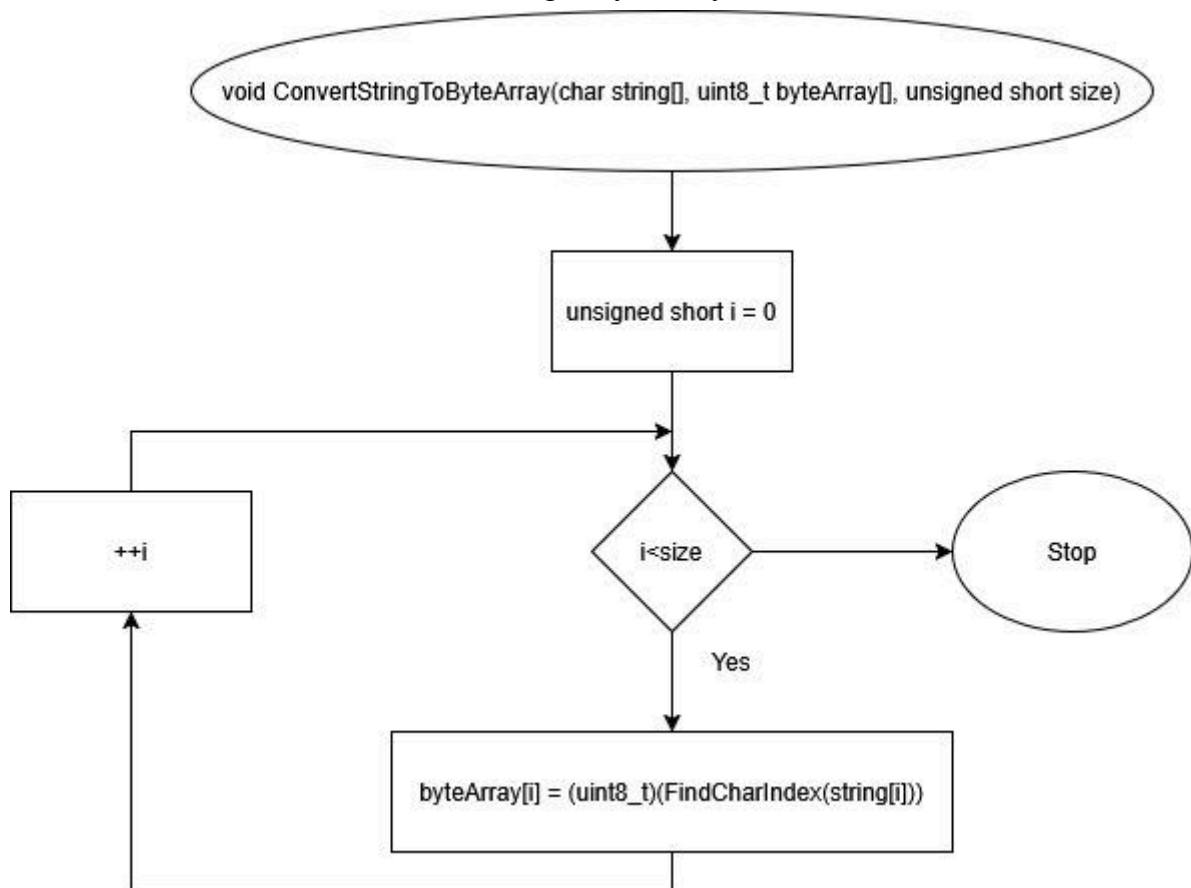
3.2.1. Flowchart FindCharIndex



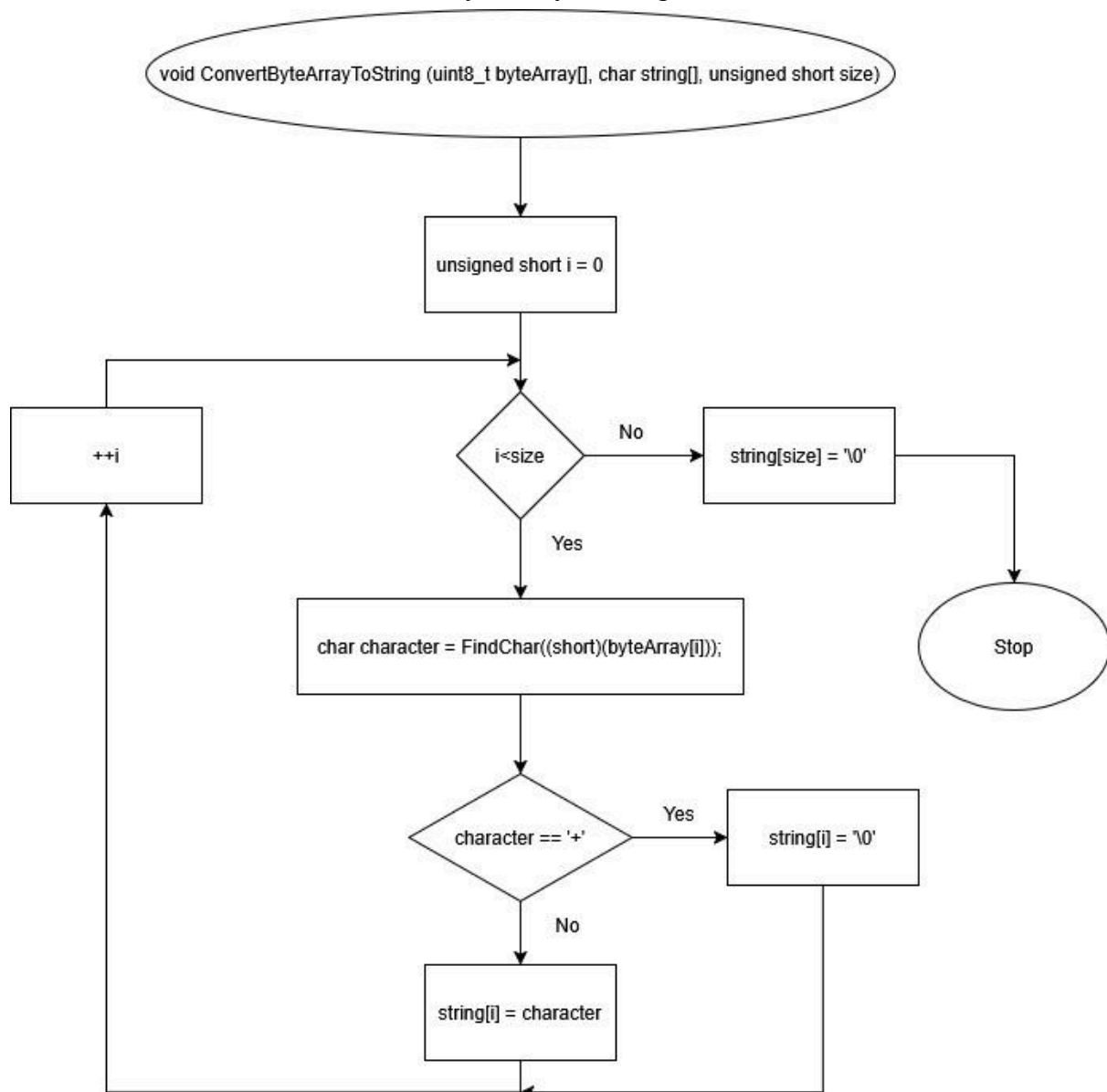
3.2.2. Flowchart FindChar



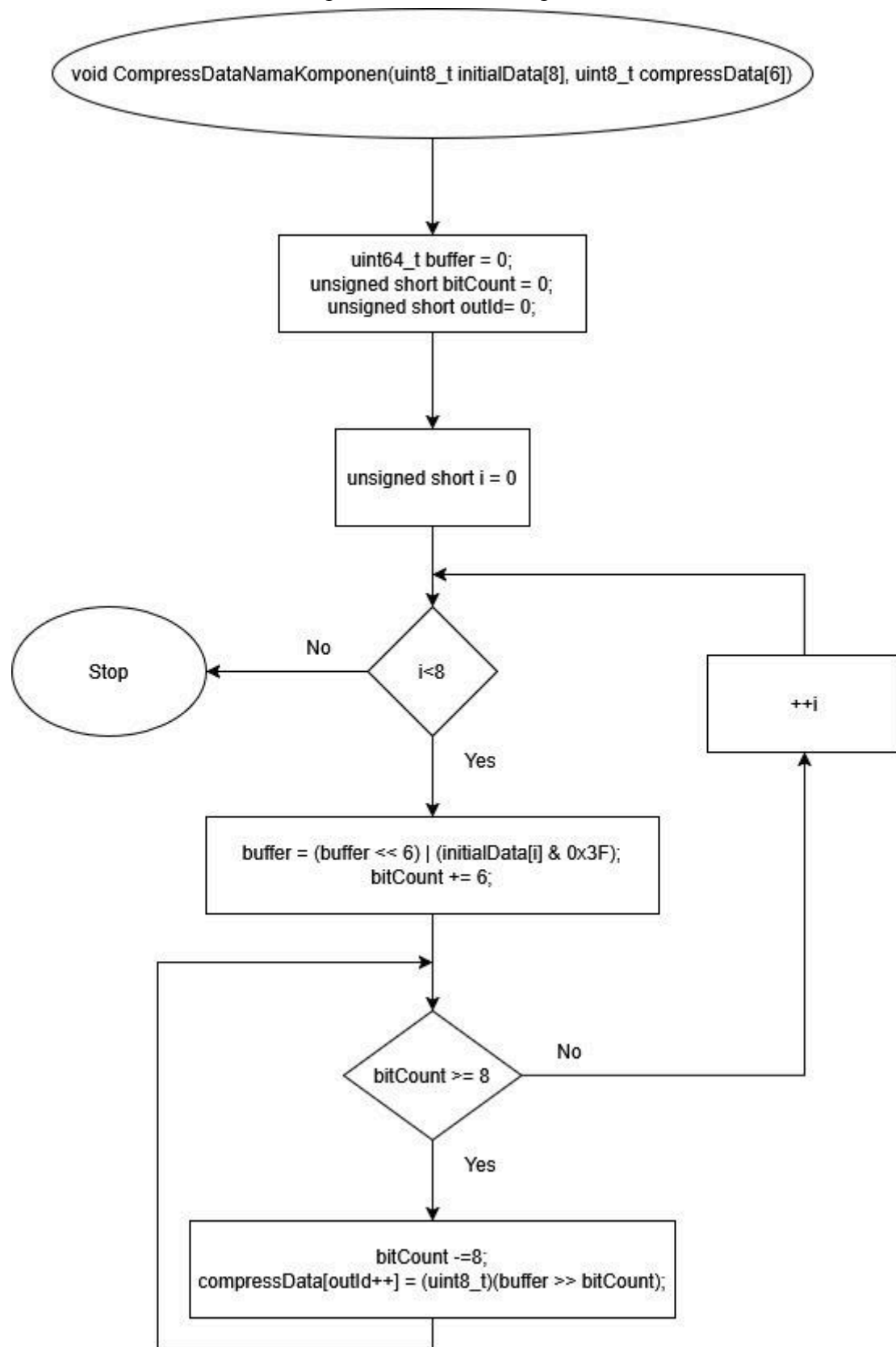
3.2.3. Flowchart ConvertStringToByteArray



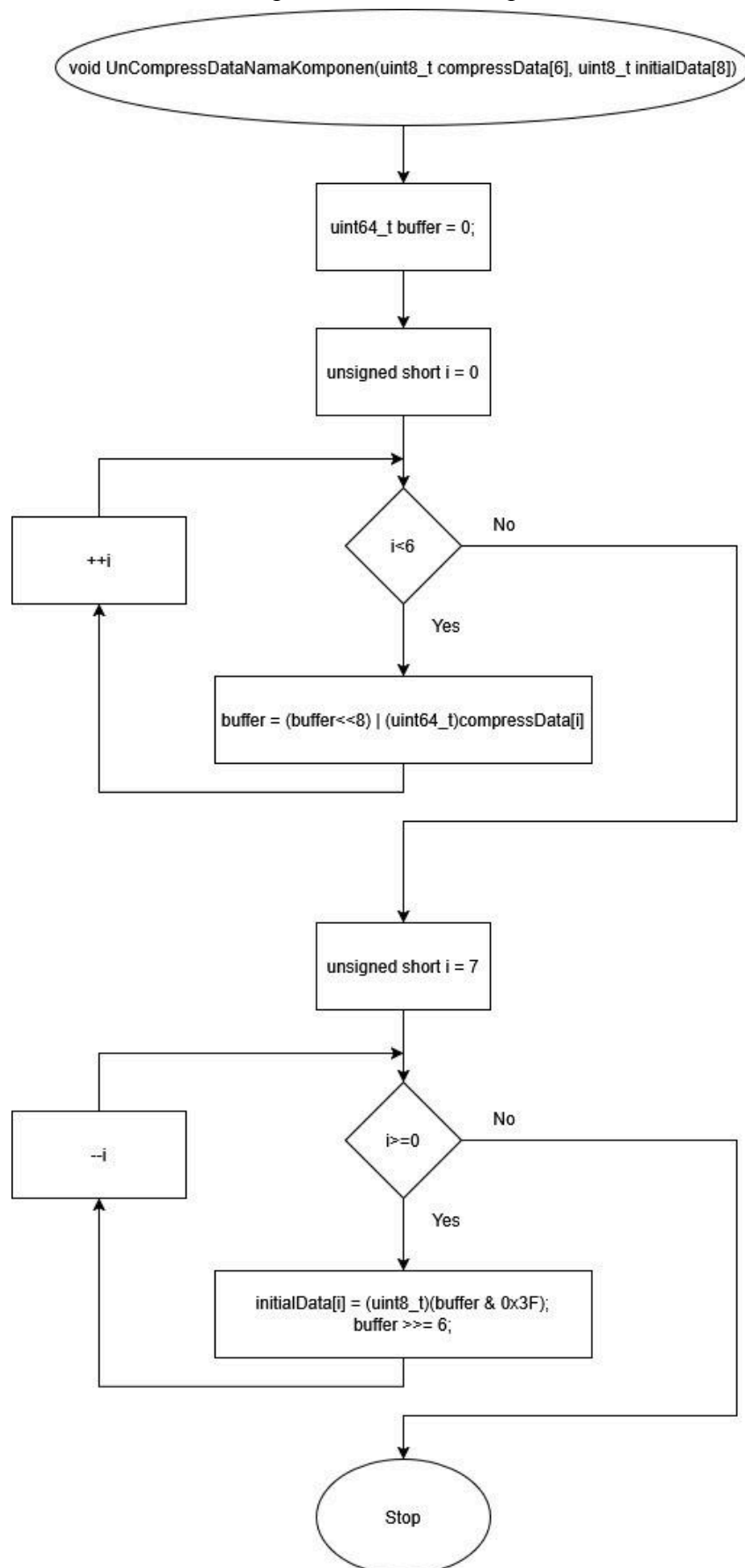
3.2.4. Flowchart ConvertByteArrayToString



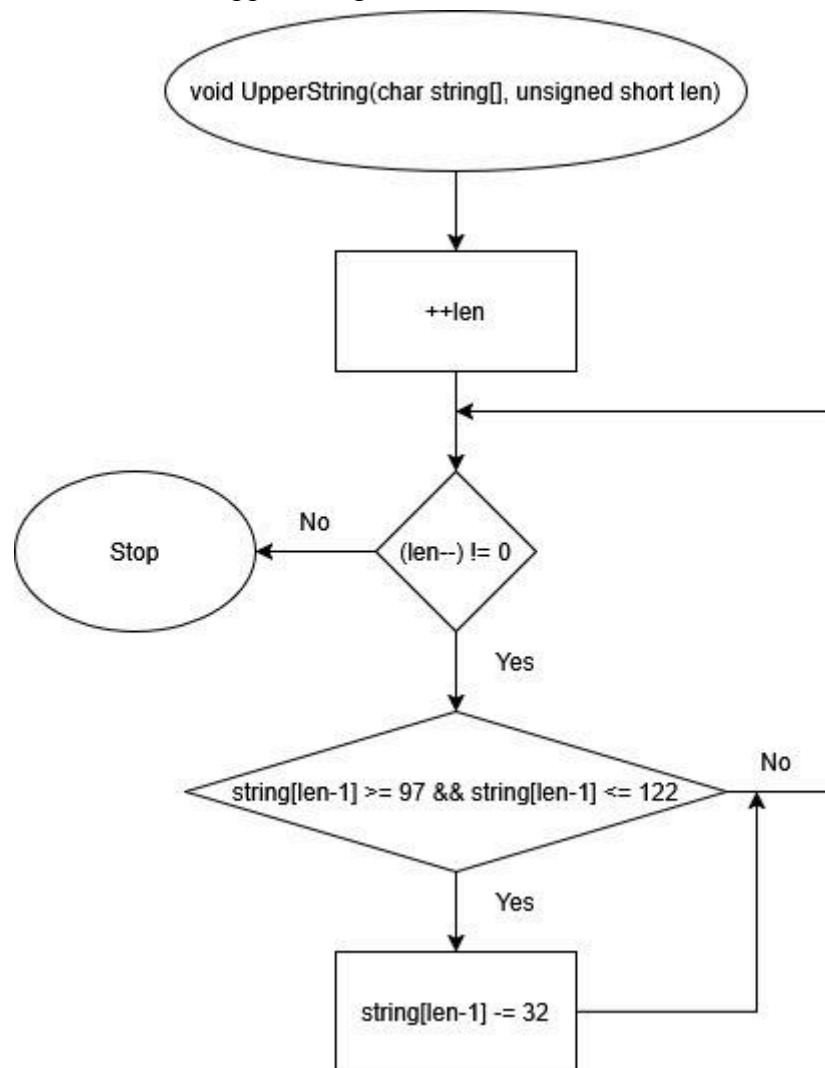
3.2.5. Flowchart CompressDataNamaKomponen



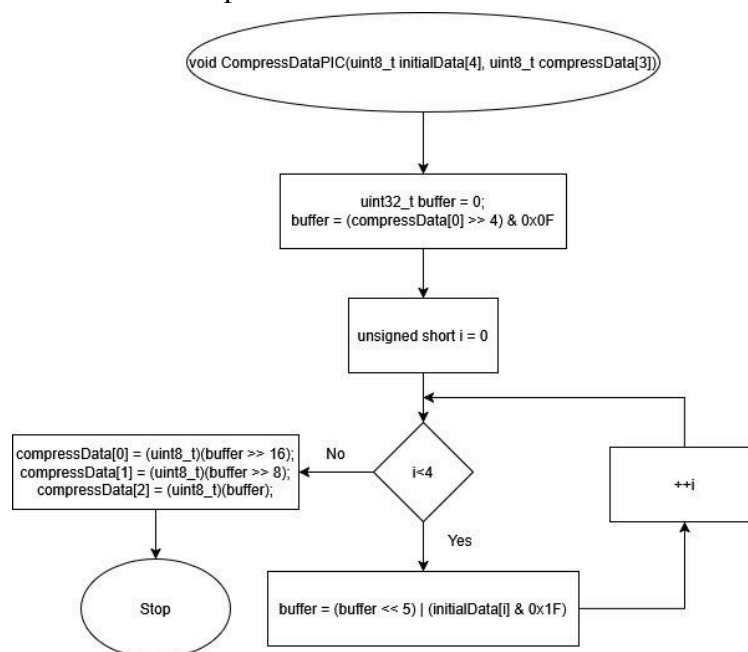
3.2.6. Flowchart UnCompressDataNamaKomponen



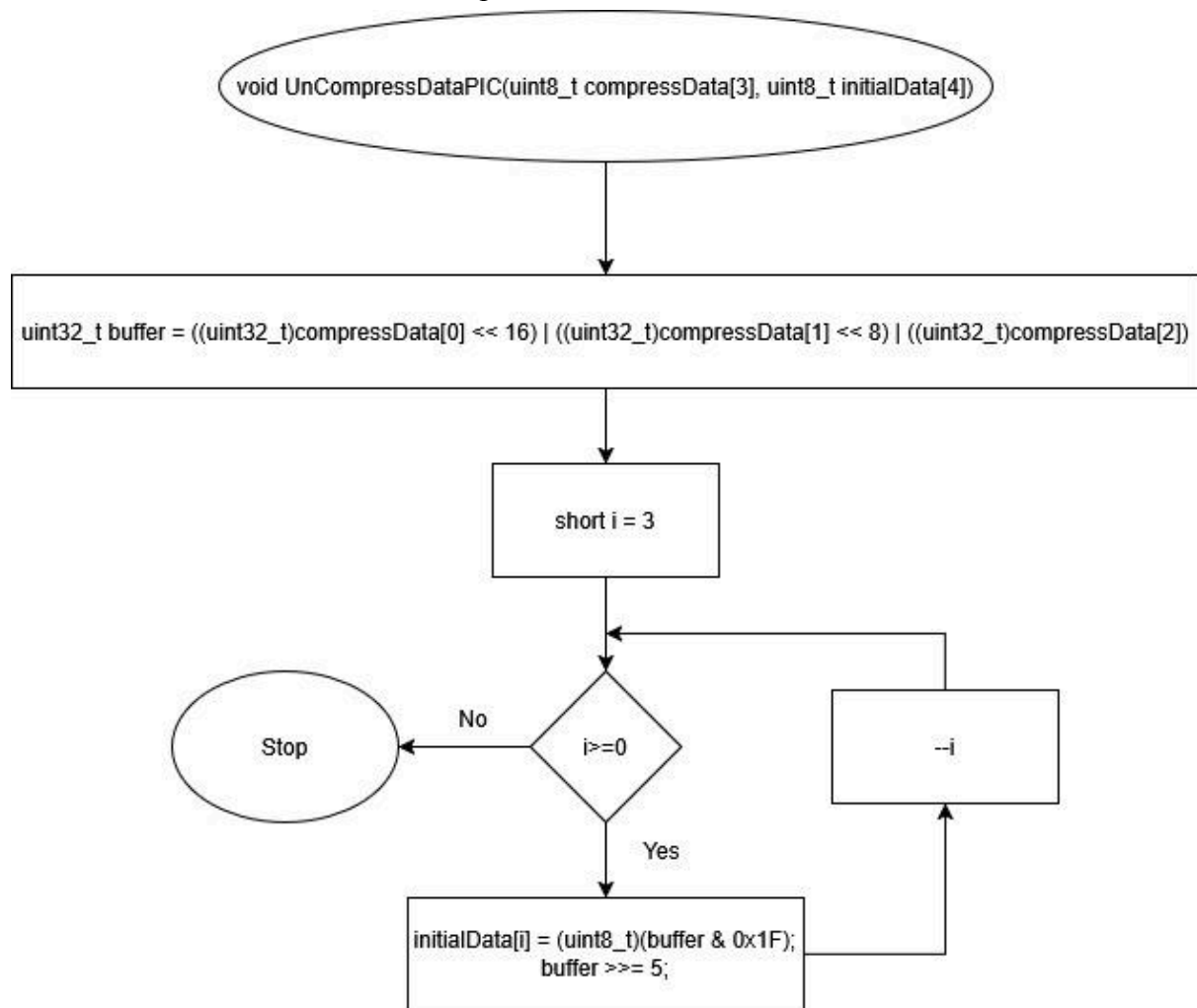
3.2.7. Flowchart UpperString



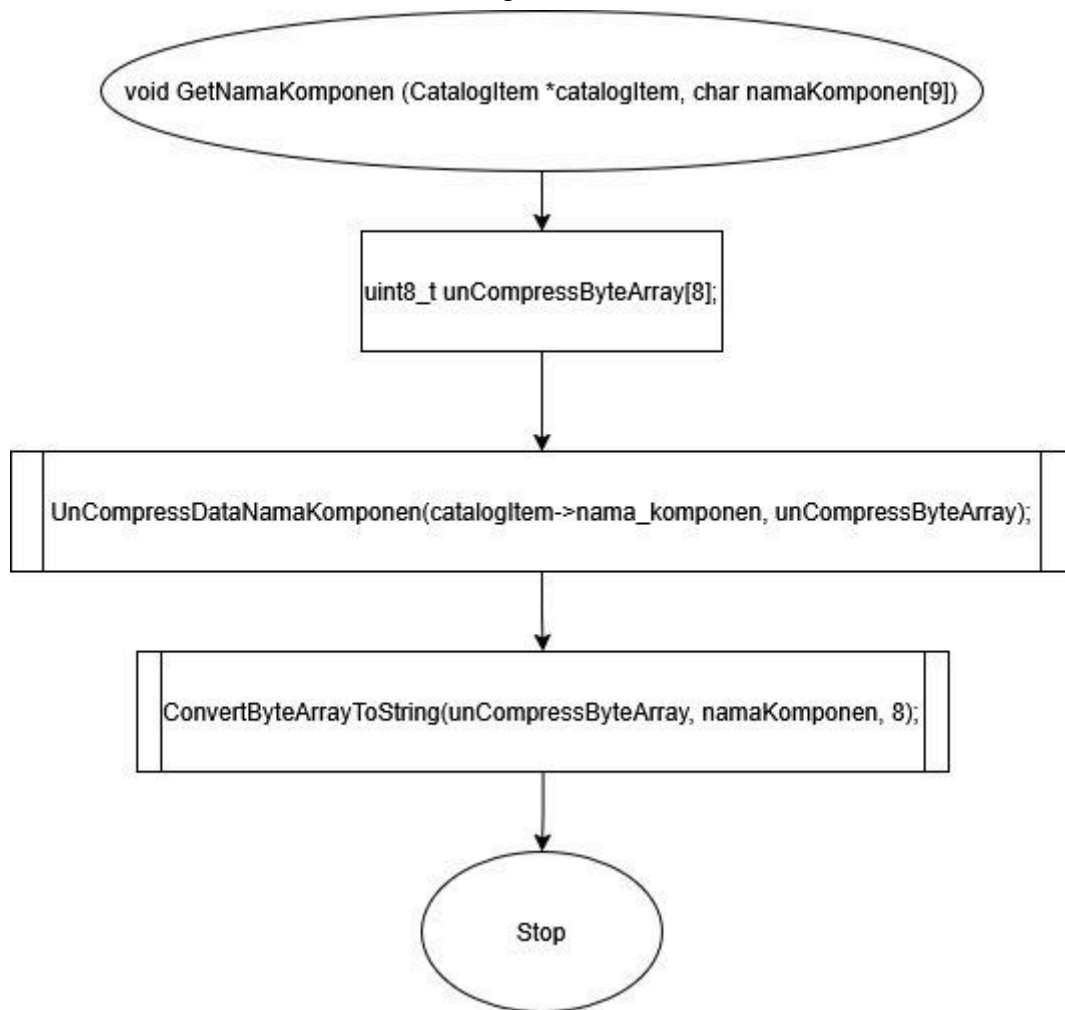
3.2.8. Flowchart CompressDataPIC



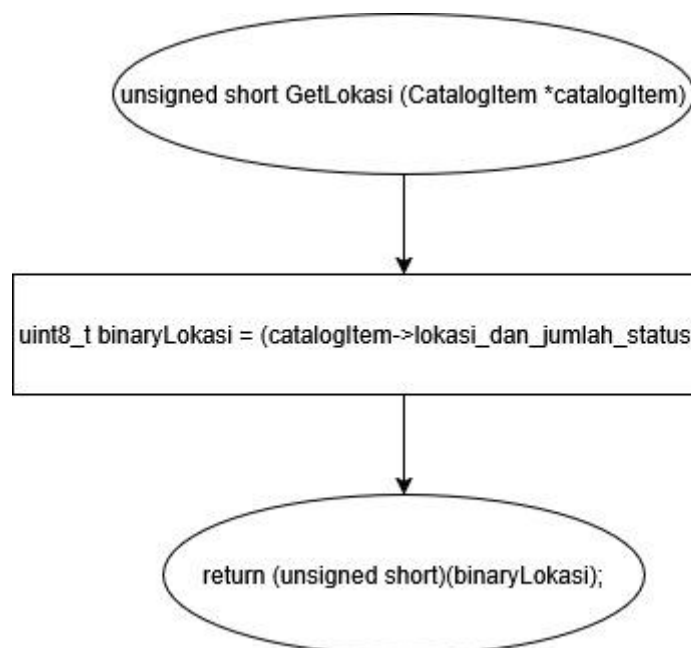
3.2.9. Flowchart UnCompressDataPIC



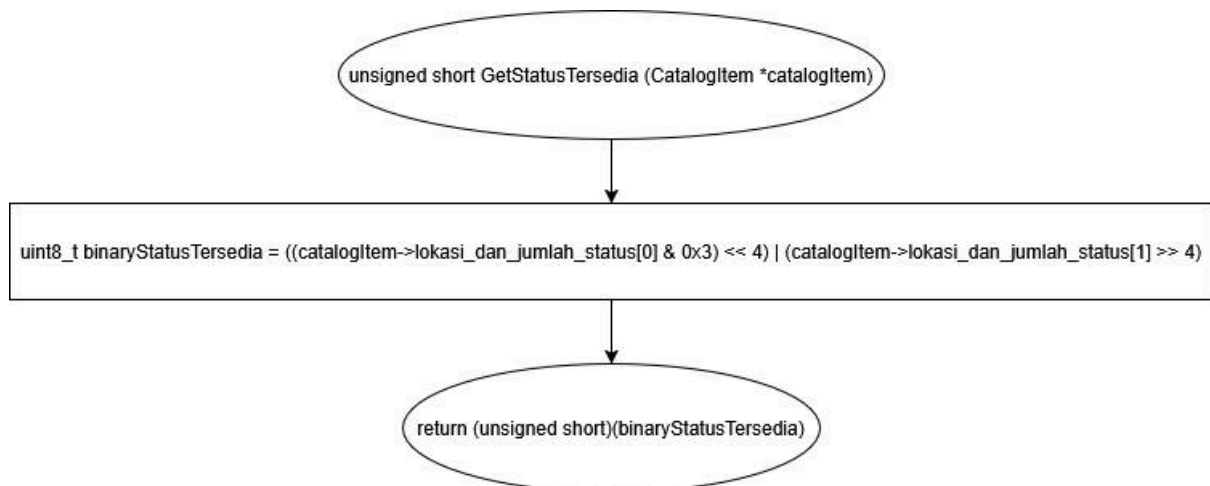
3.2.10. Flowchart GetNamaKomponen



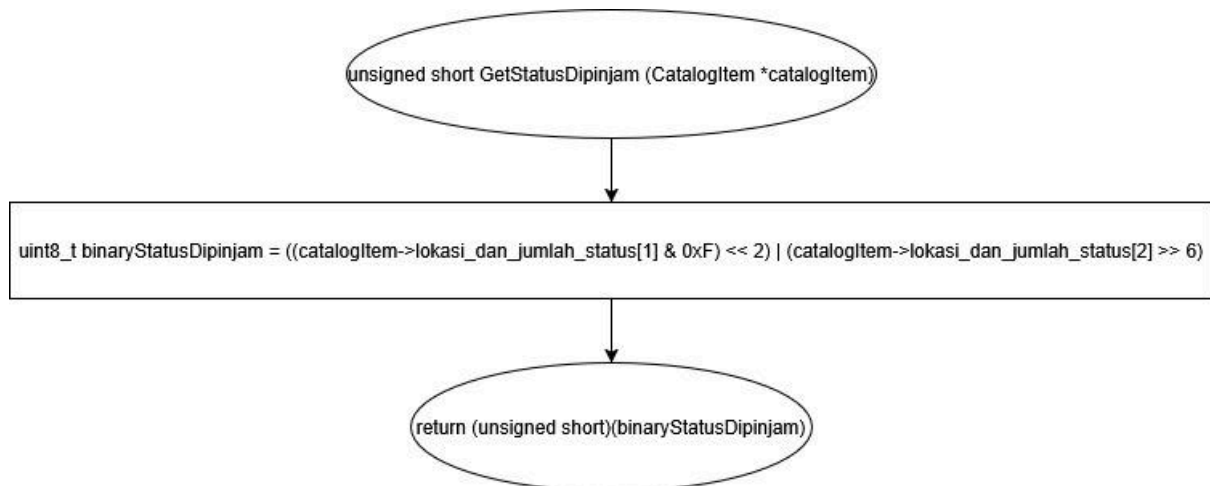
3.2.11. Flowchart GetLokasi



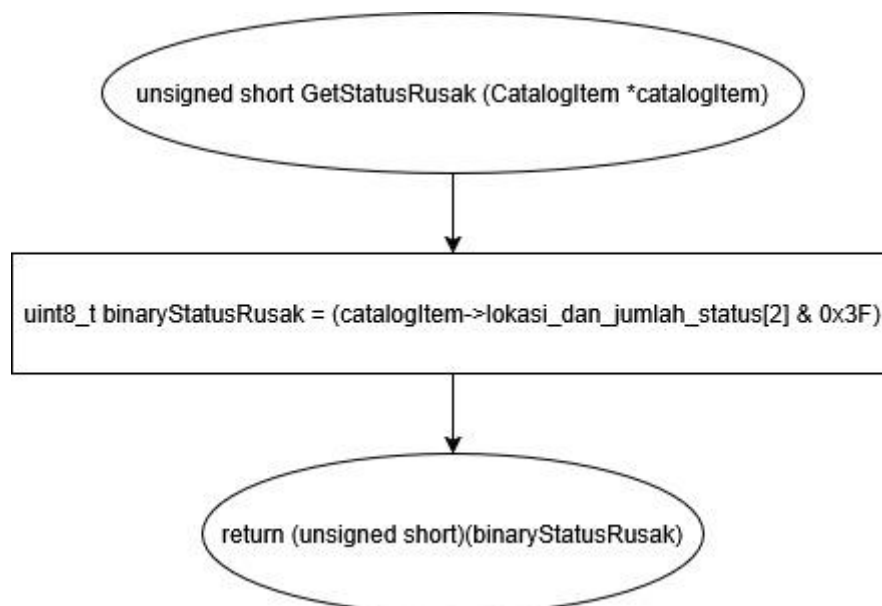
3.2.12. Flowchart GetStatusTersedia



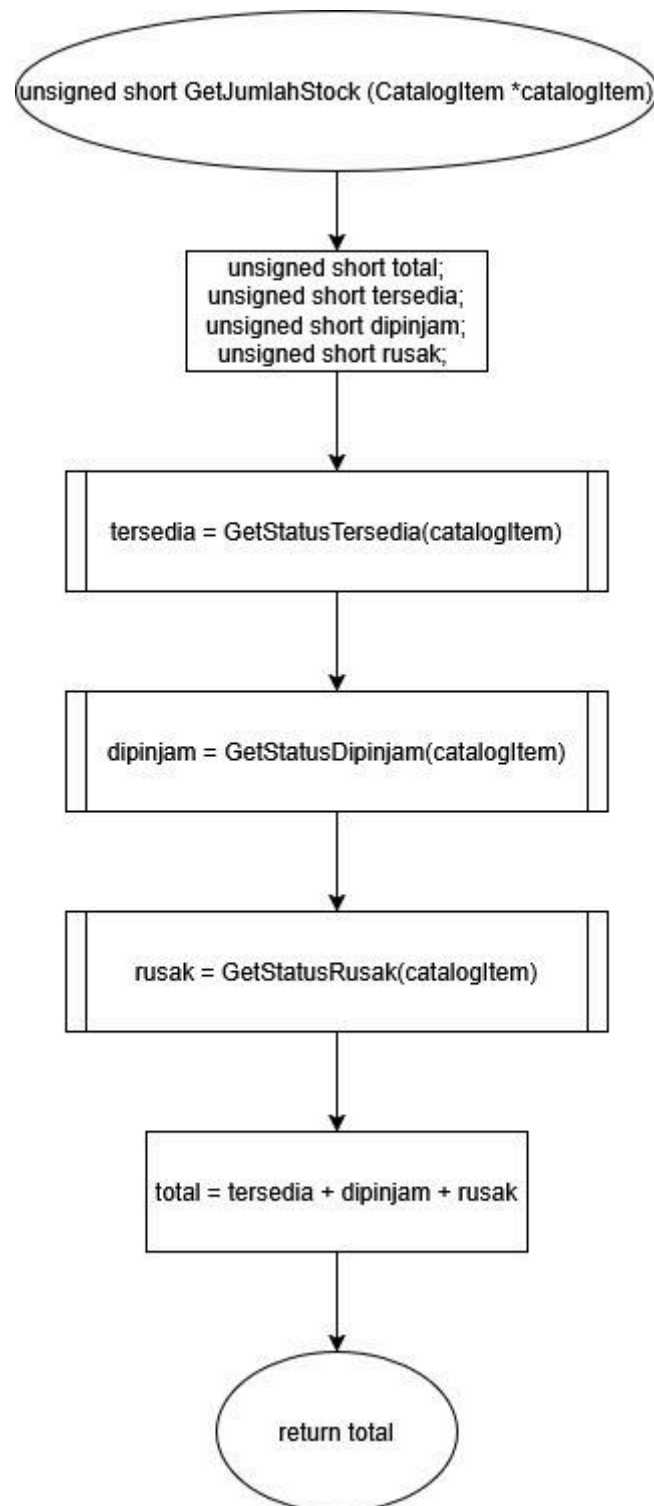
3.2.13. Flowchart GetStatusDipinjam



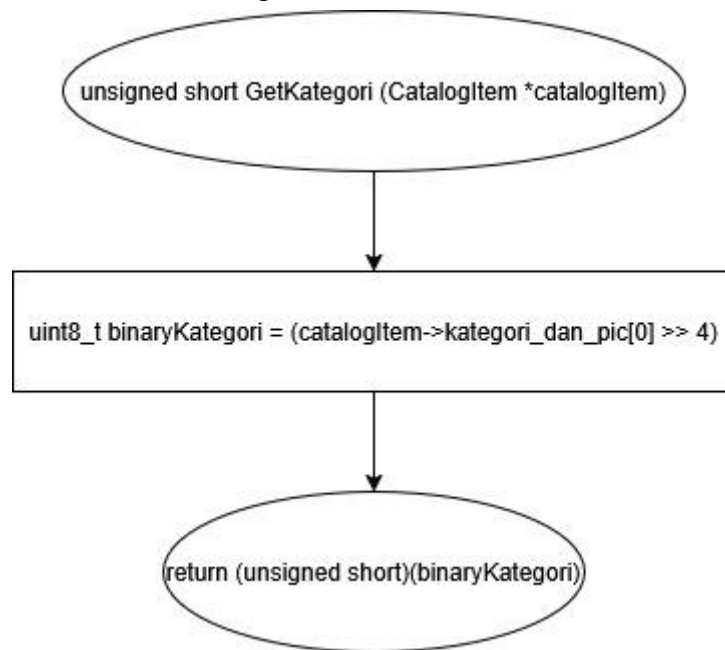
3.2.14. Flowchart GetStatusRusak



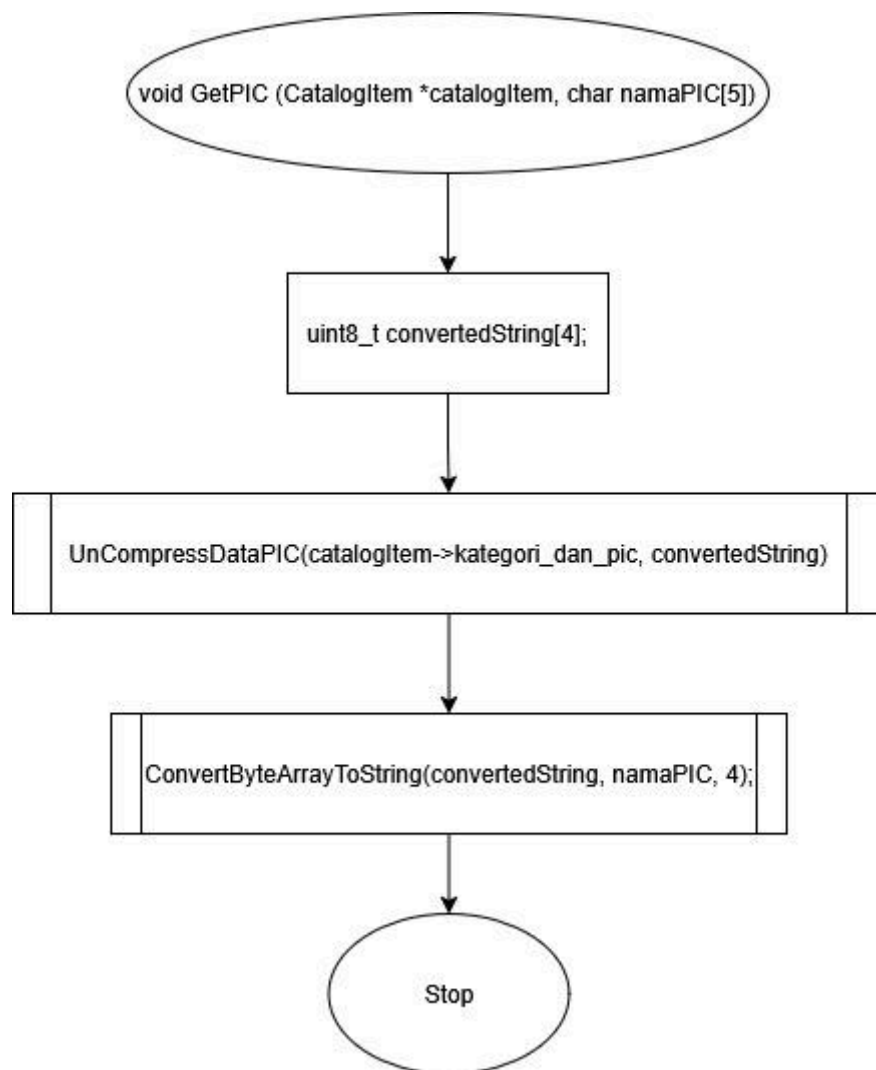
3.2.15. Flowchart GetJumlahStock



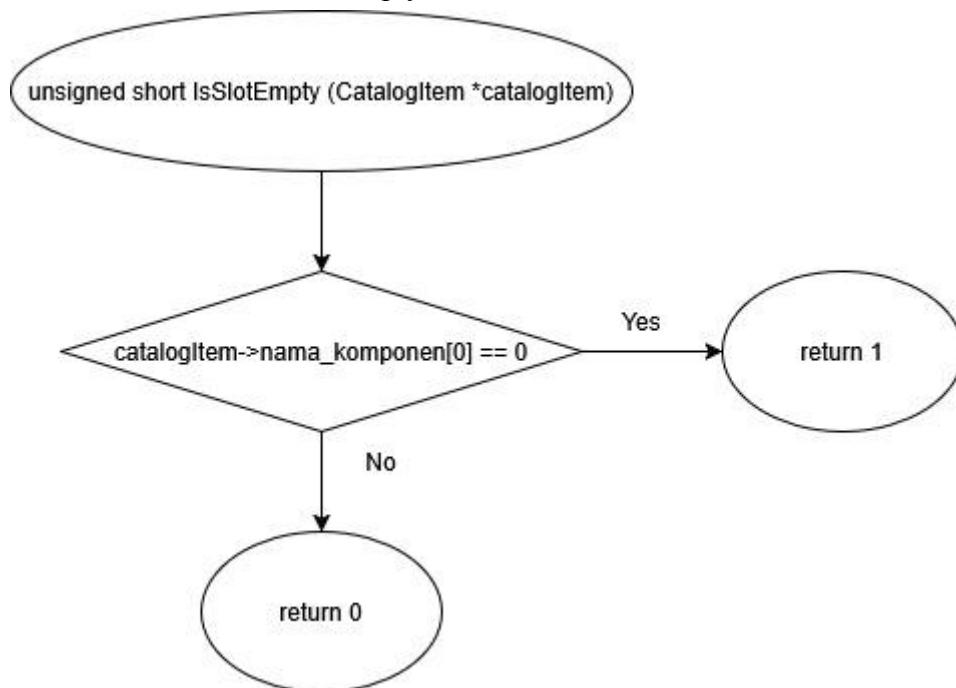
3.2.16. Flowchart GetKategori



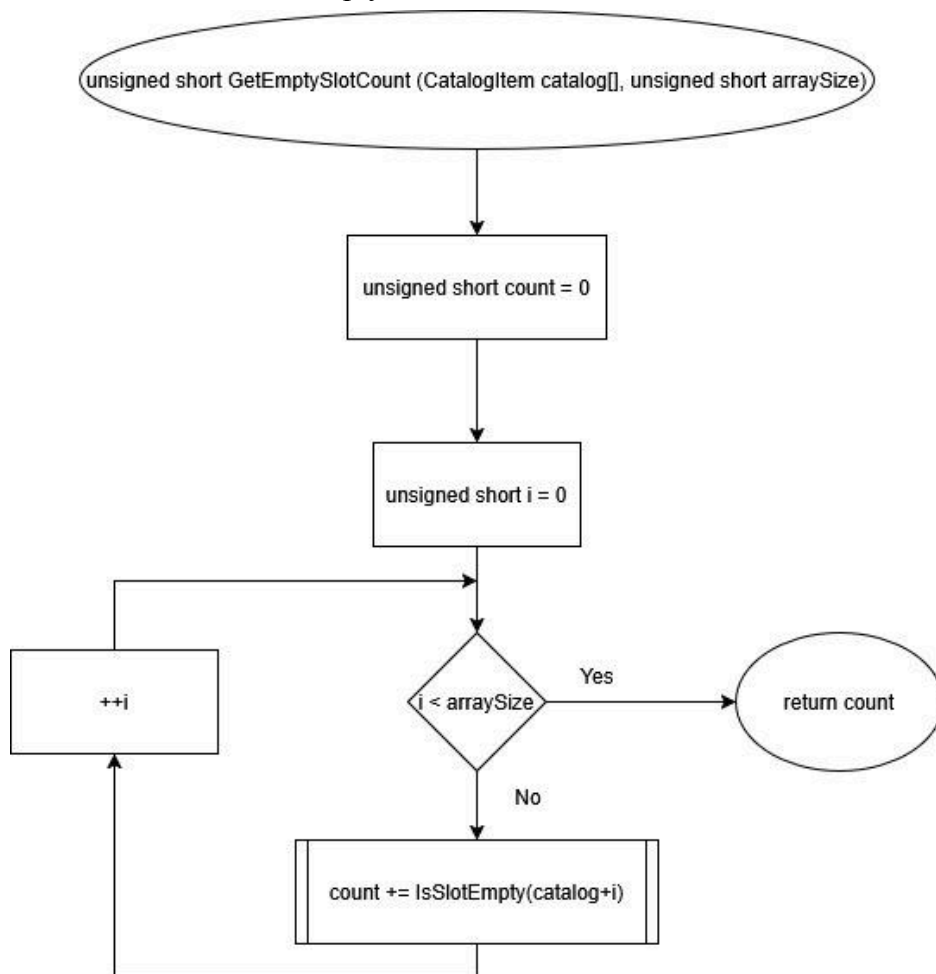
3.2.17. Flowchart GetPIC



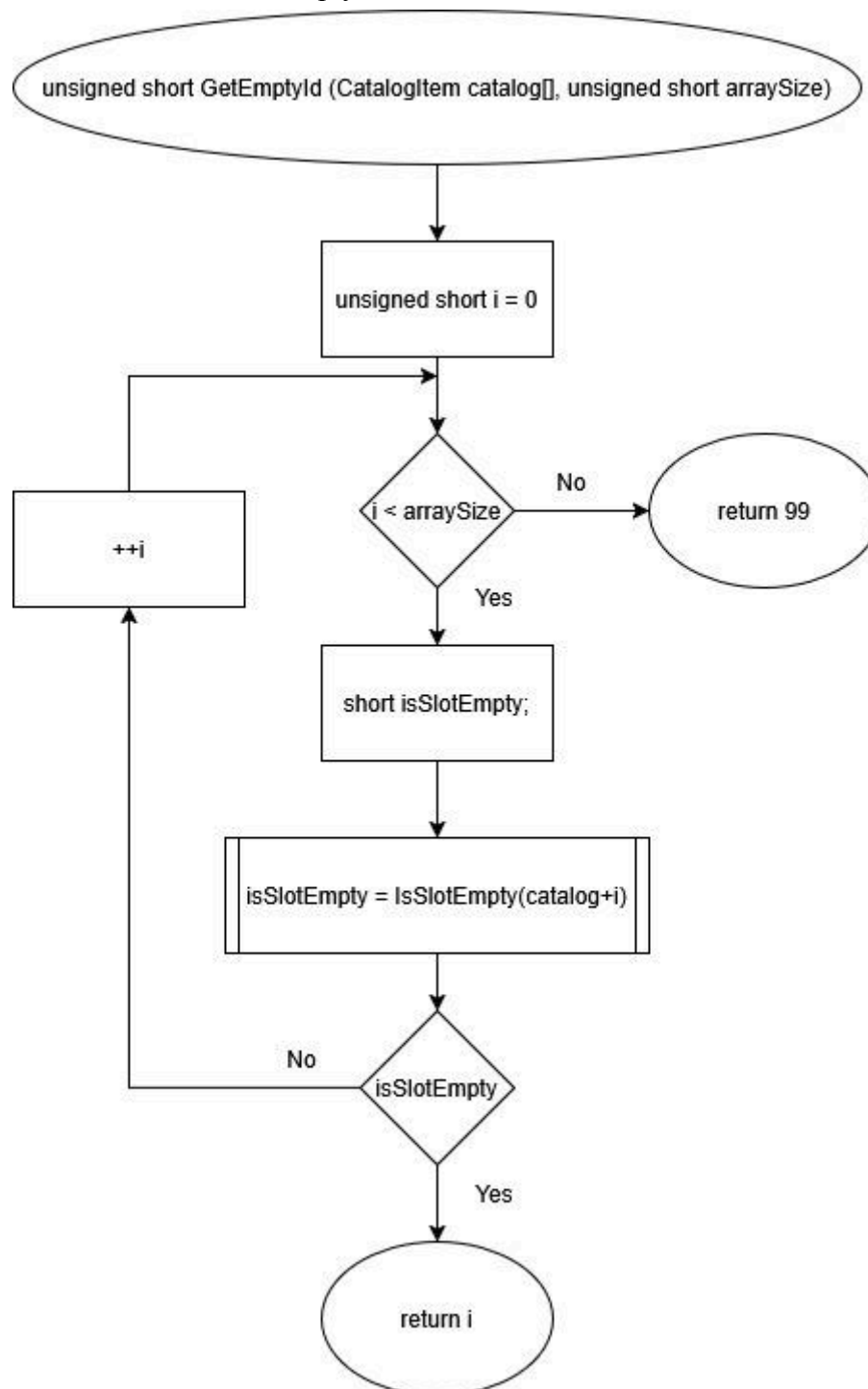
3.2.18. Flowchart IsSlotEmpty



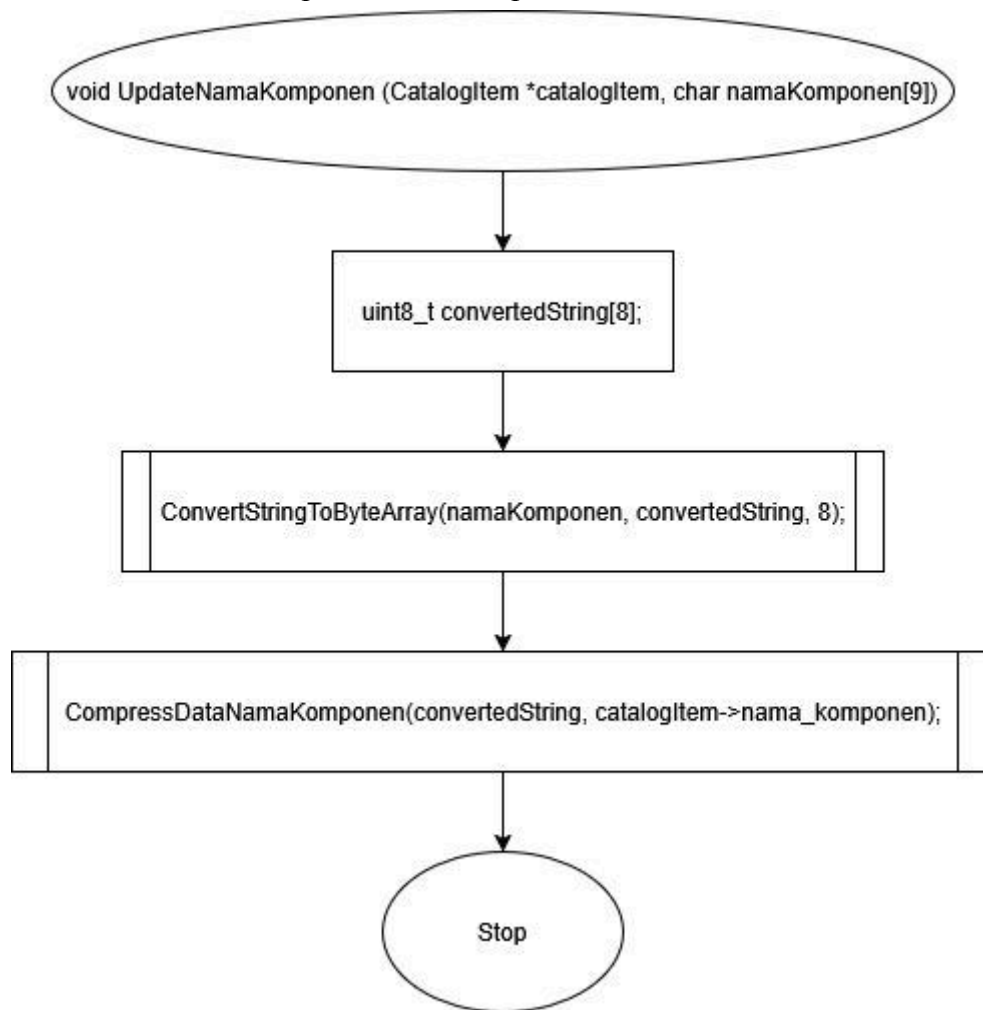
3.2.19. Flowchart GetEmptySlotCount



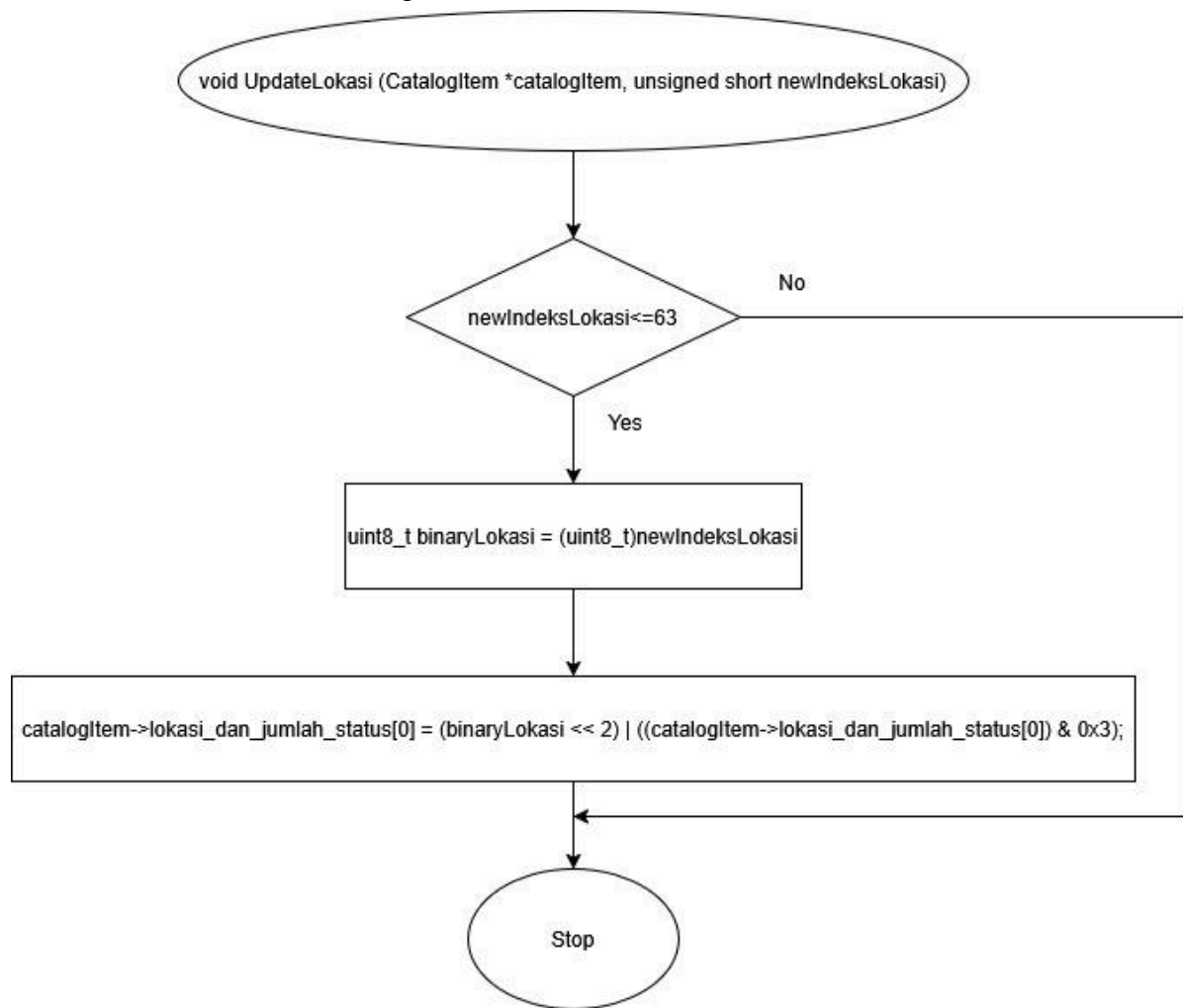
3.2.20. Flowchart GetEmptyId



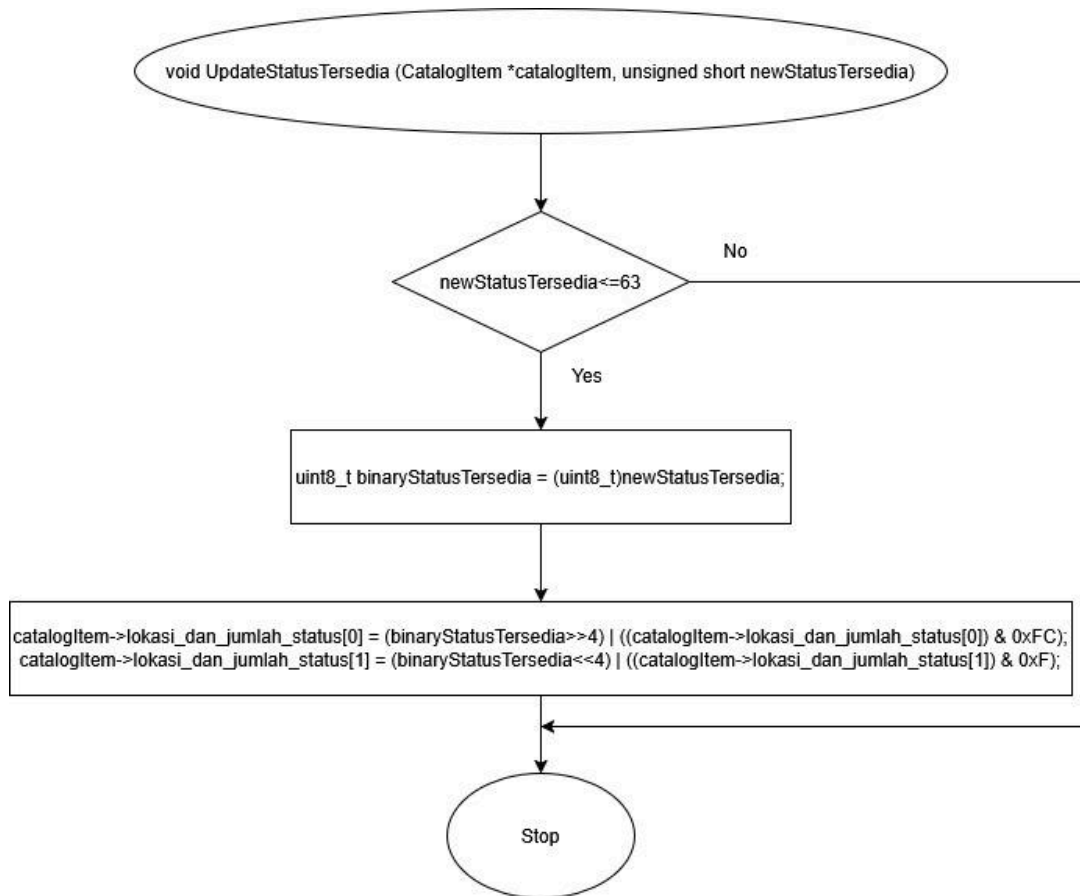
3.2.21. Flowchart UpdateNamaKomponen



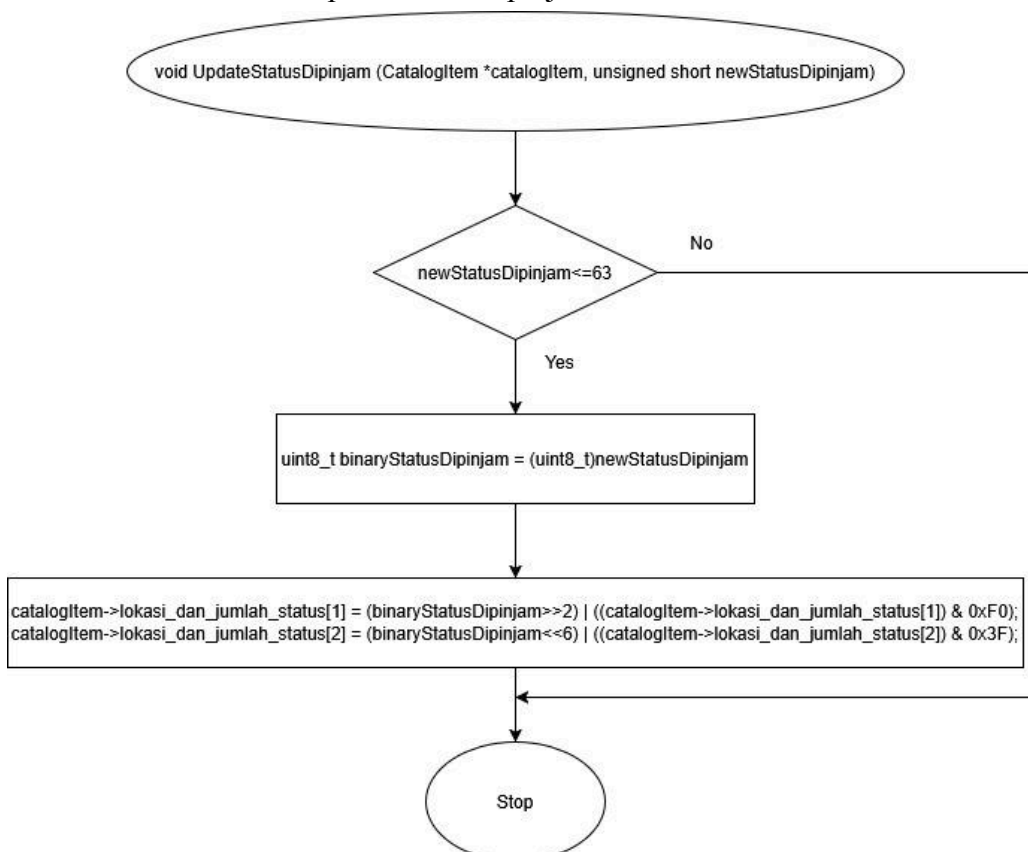
3.2.22. Flowchart UpdateLokasi



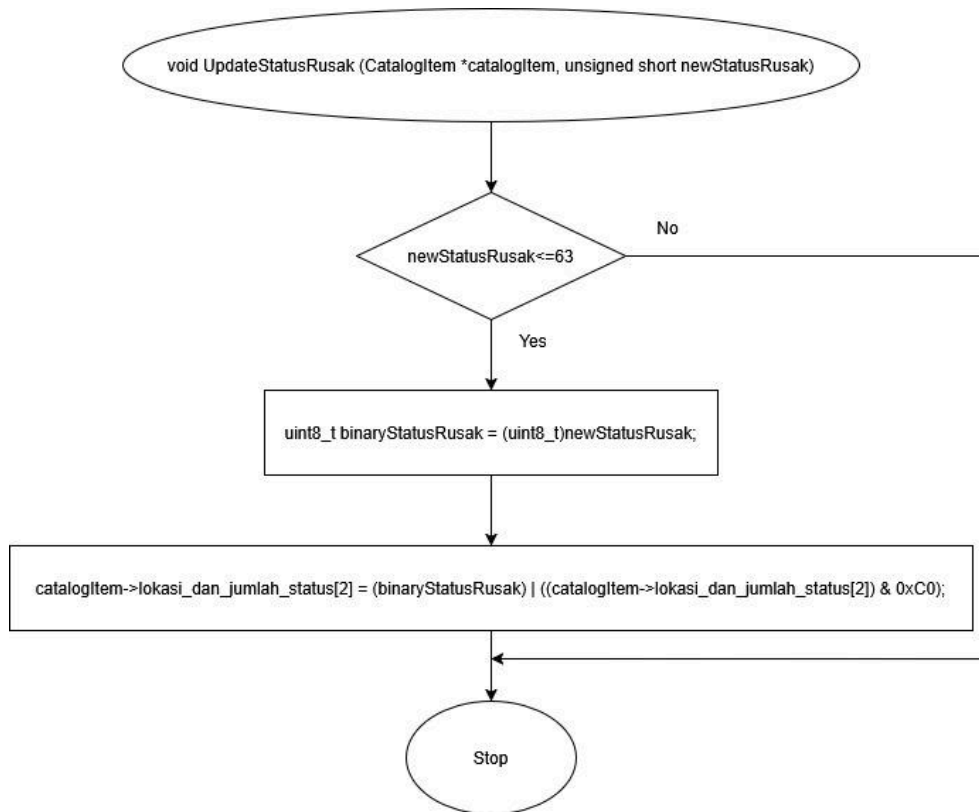
3.2.23. Flowchart UpdateStatusTersedia



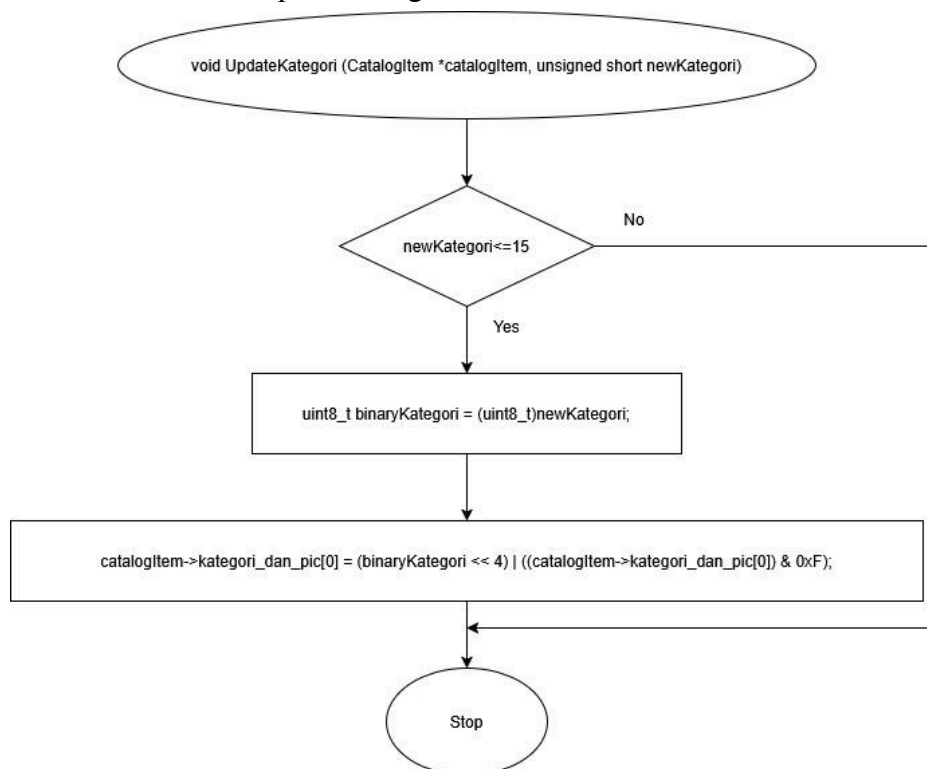
3.2.24. Flowchart UpdateStatusDipinjam



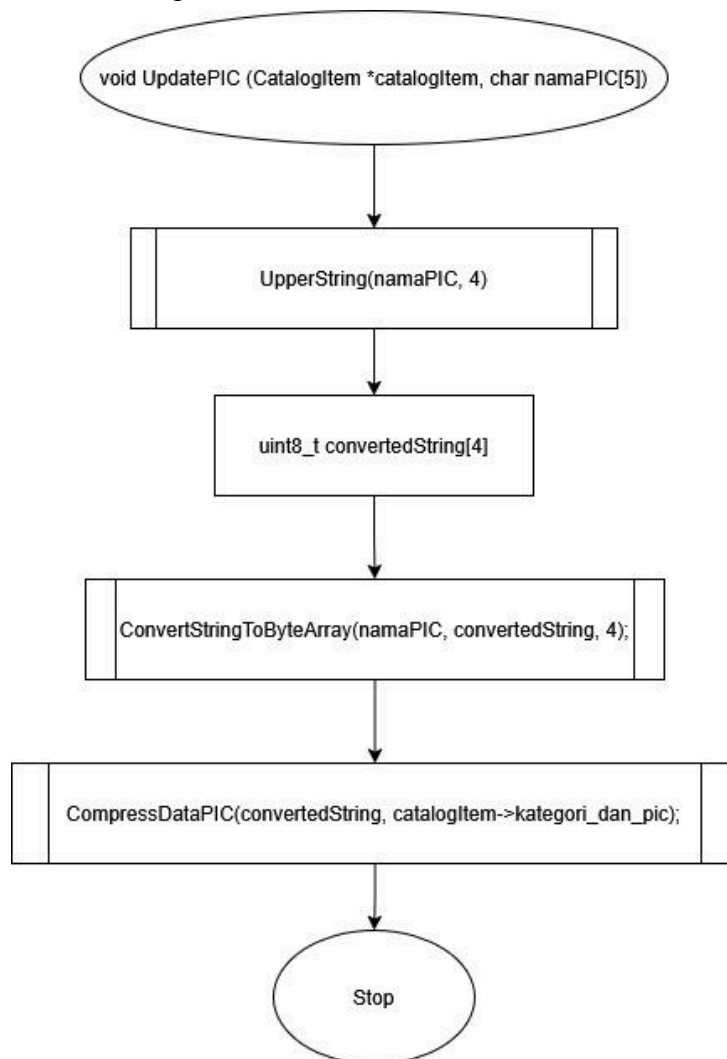
3.2.25. Flowchart UpdateStatusRusak



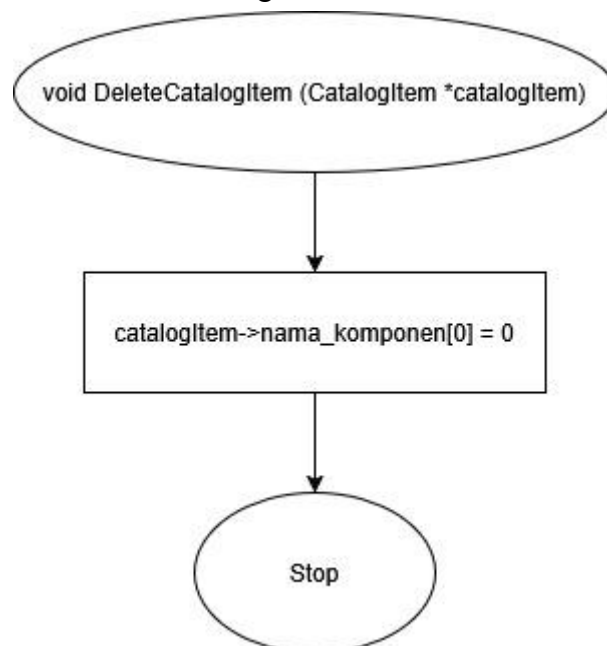
3.2.26. Flowchart UpdateKategori



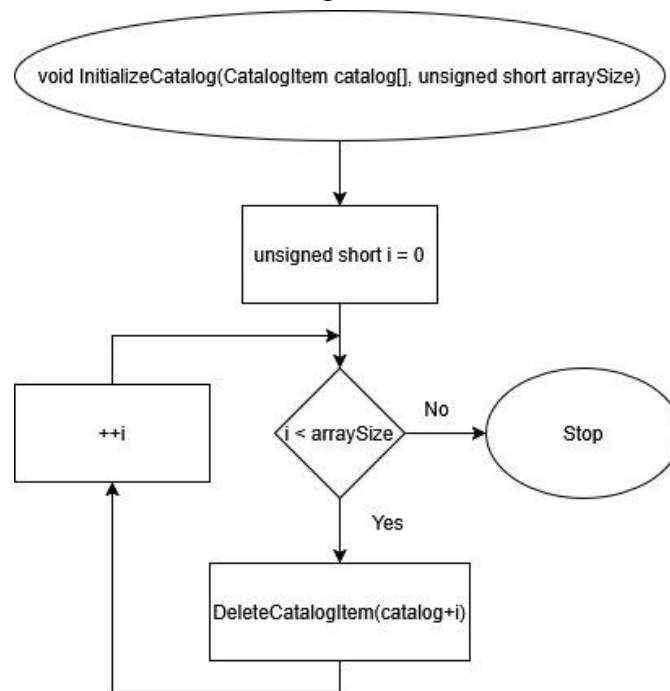
3.2.27. Flowchart UpdatePIC



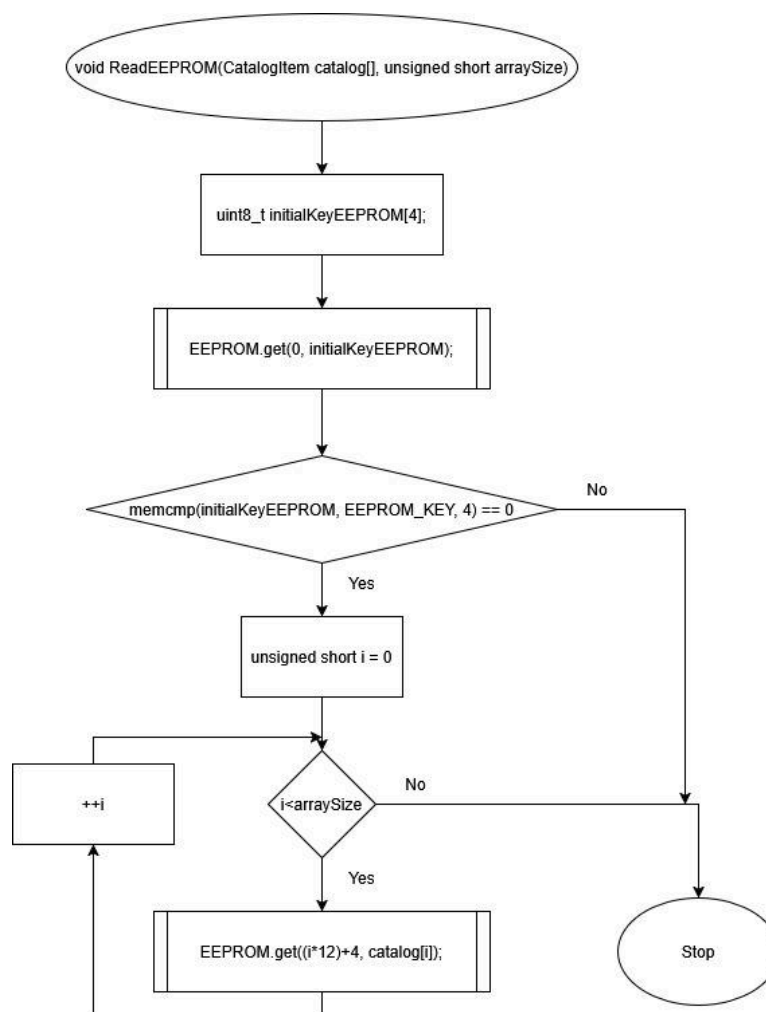
3.2.28. Flowchart DeleteCatalogItem



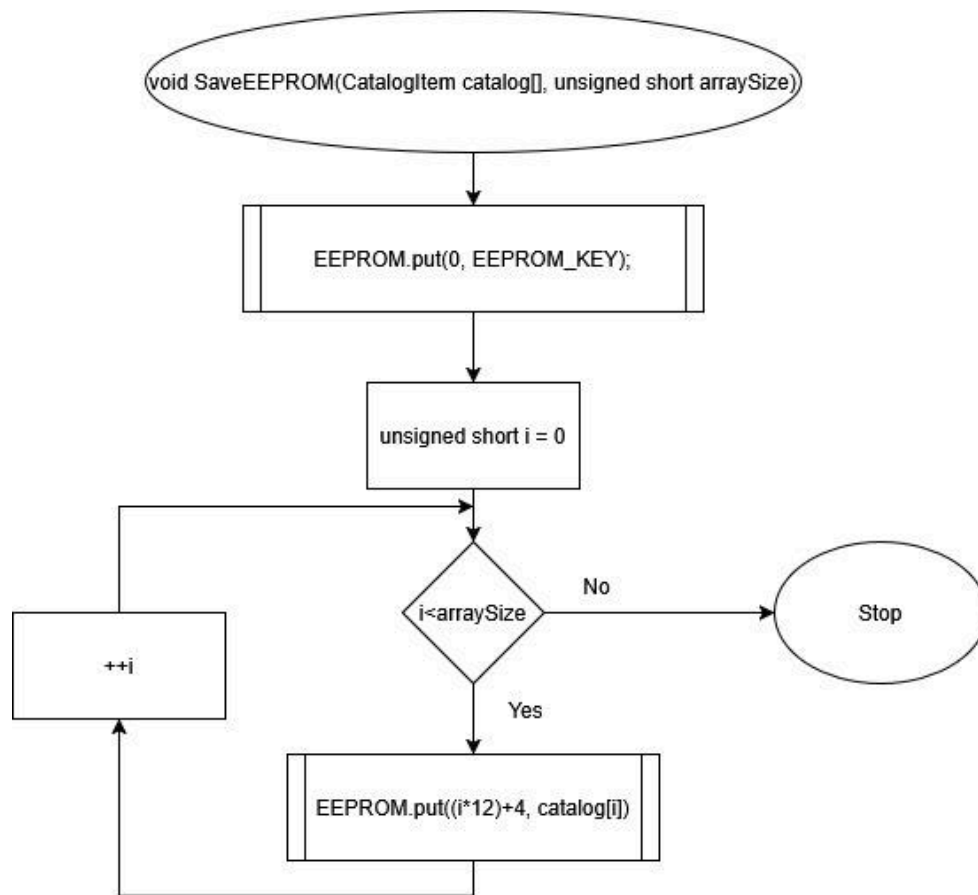
3.2.29. Flowchart InitializeCatalog



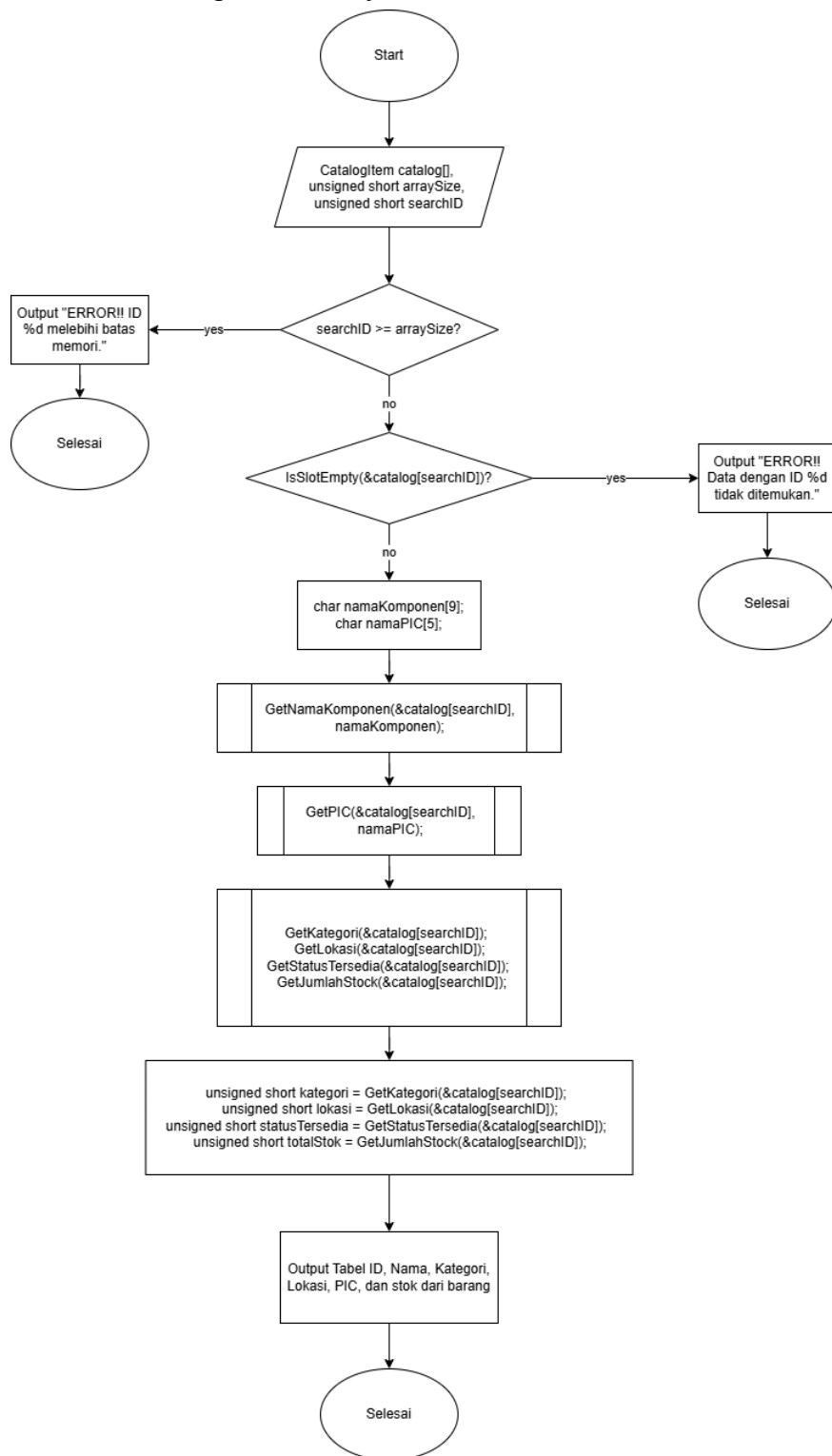
3.2.30. Flowchart ReadEEPROM



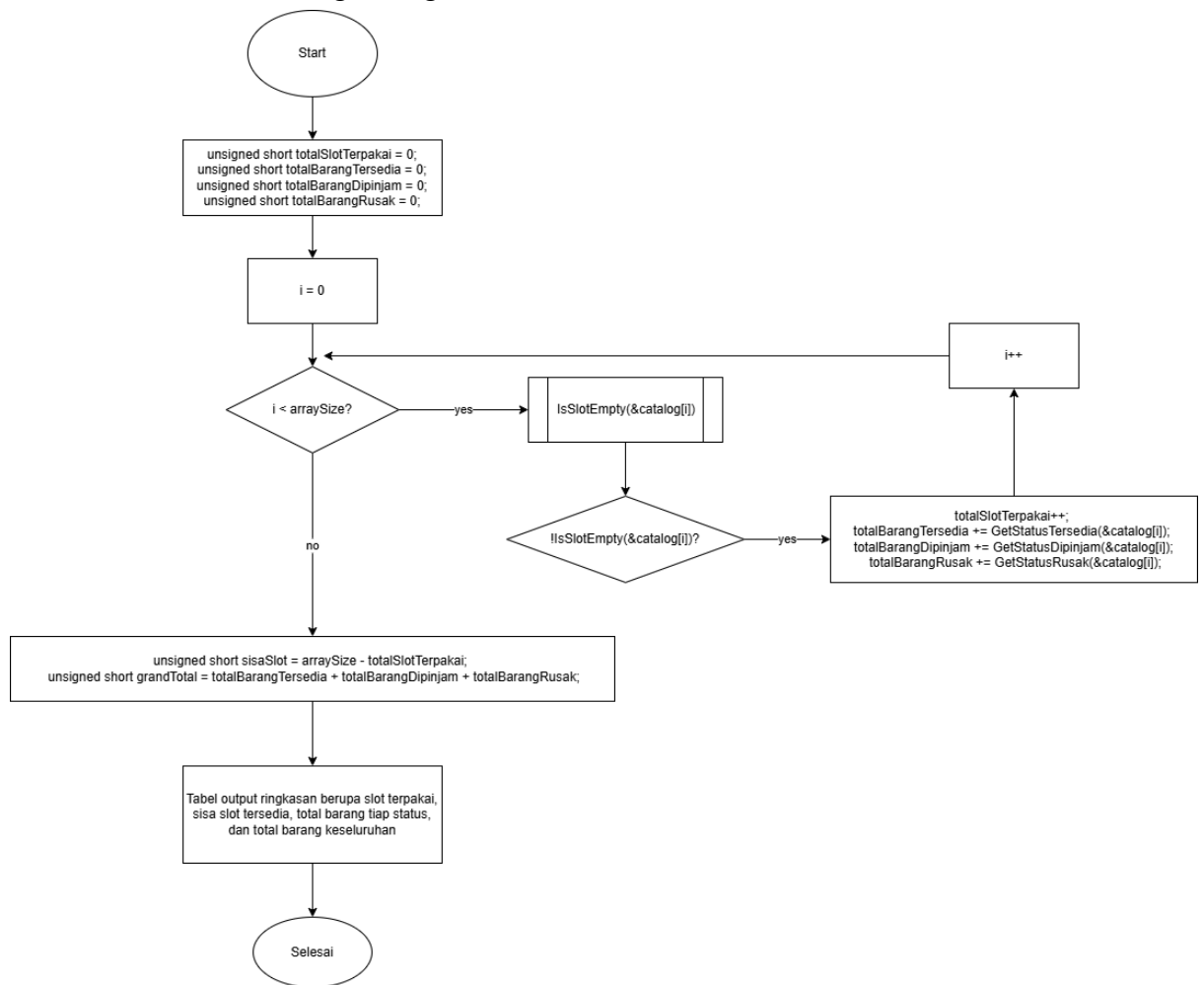
3.2.31. Flowchart SaveEEPROM



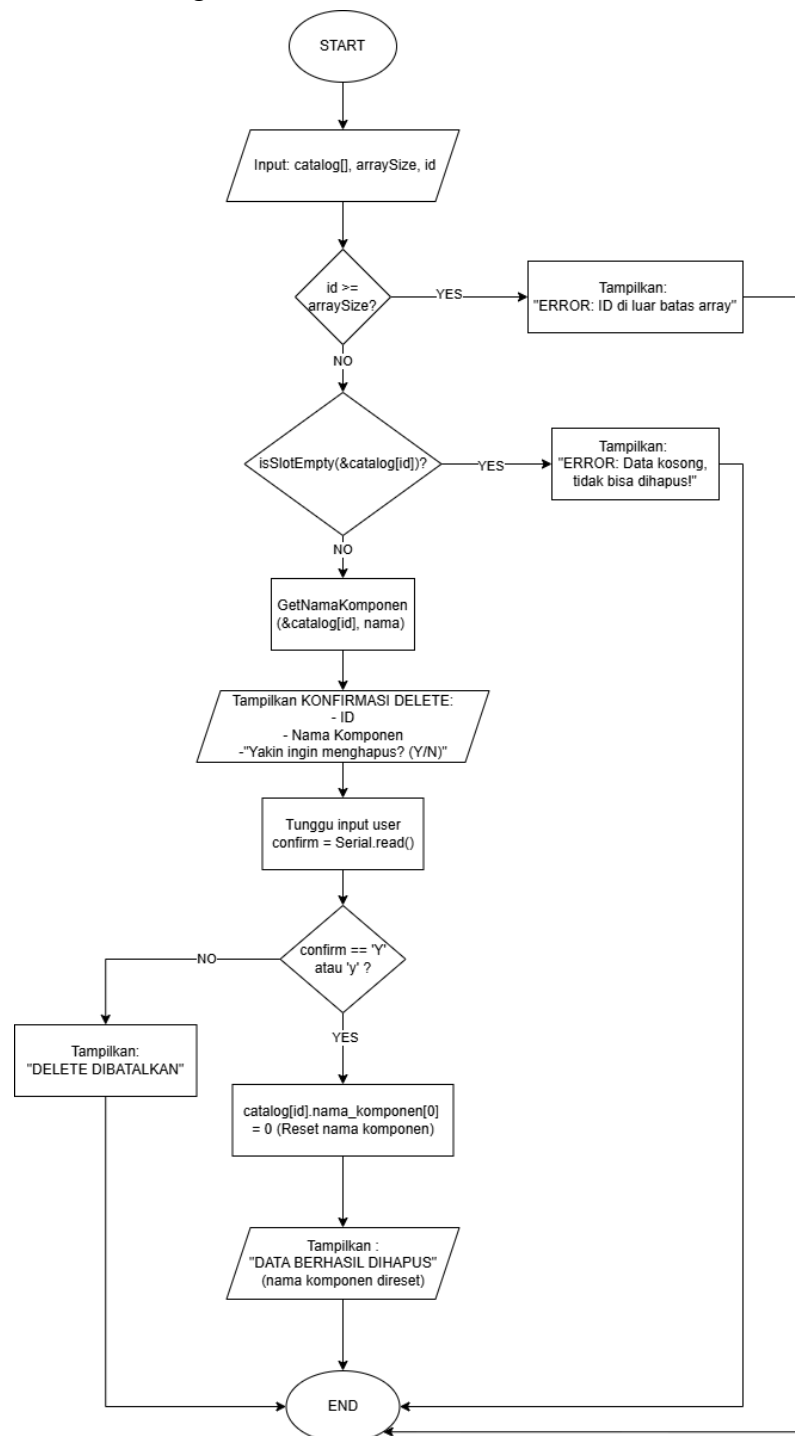
3.2.32. Flowchart Fungsi Search By ID



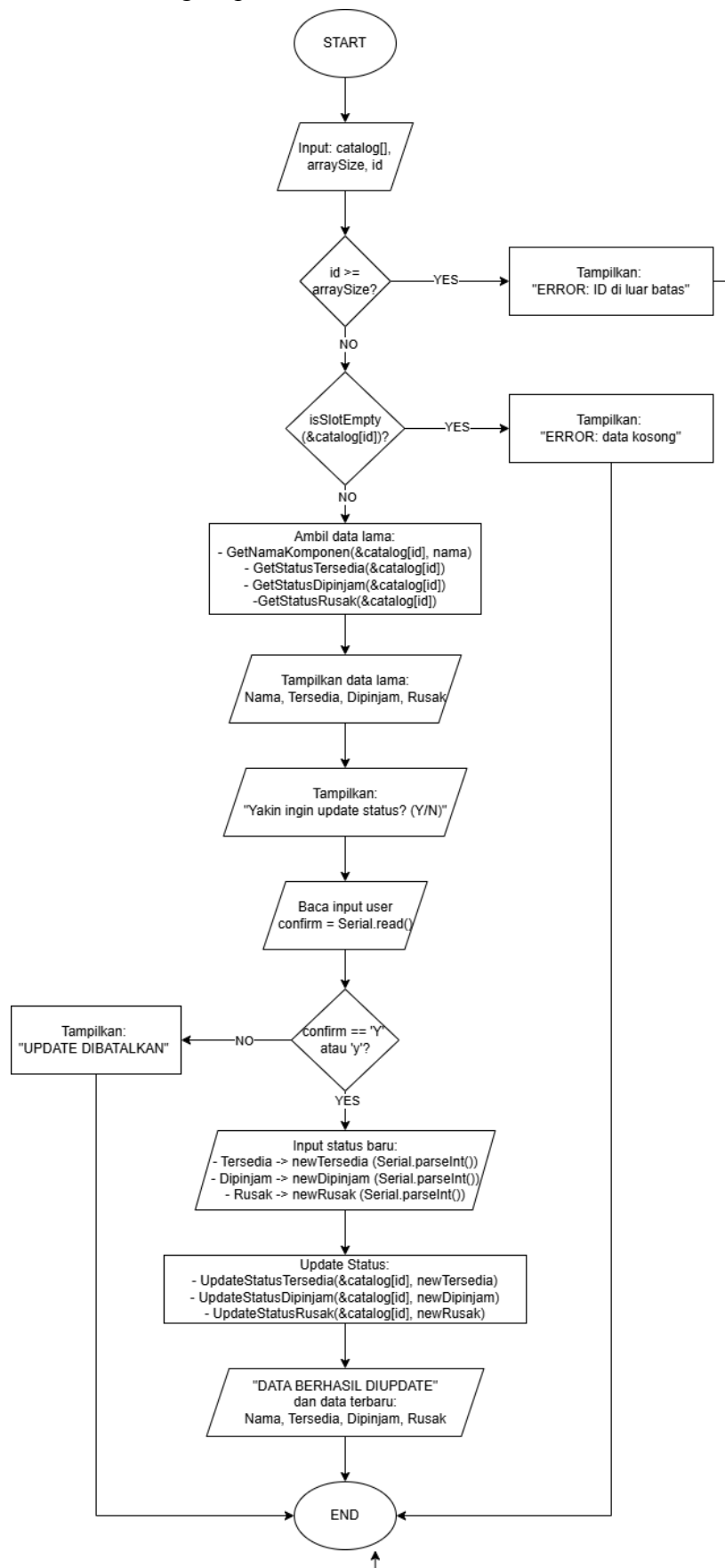
3.2.33. Flowchart Fungsi Ringkasan



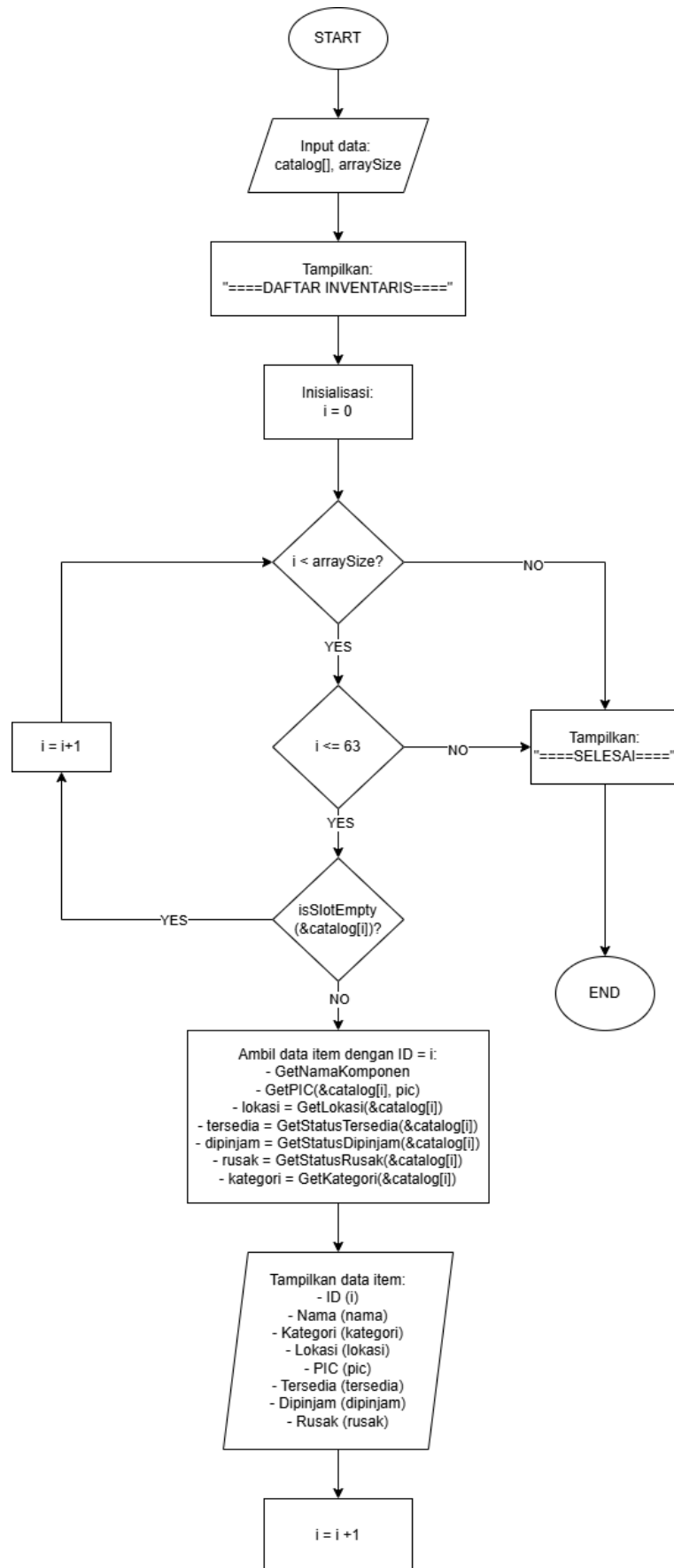
3.2.34. Flowchart Fungsi delete



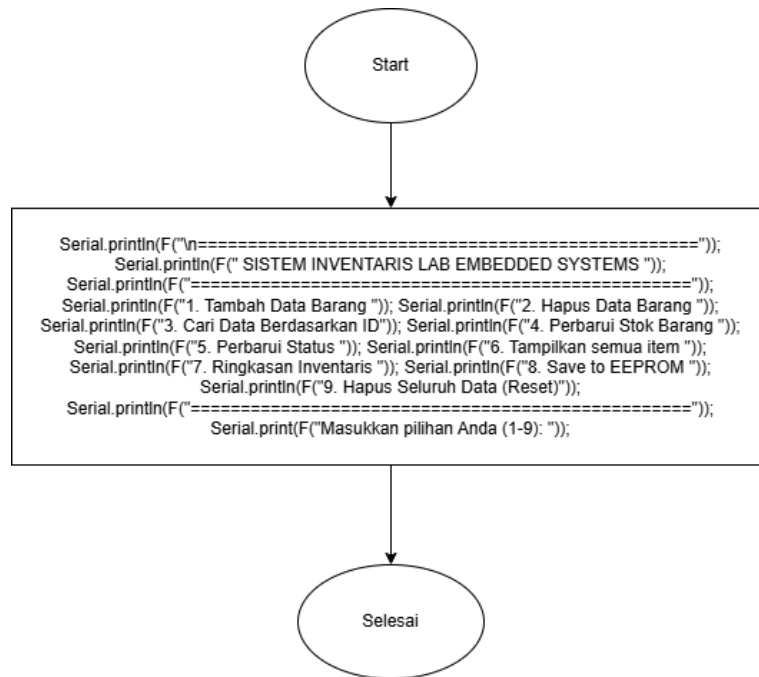
3.2.35. Flowchart Fungsi updateStatus



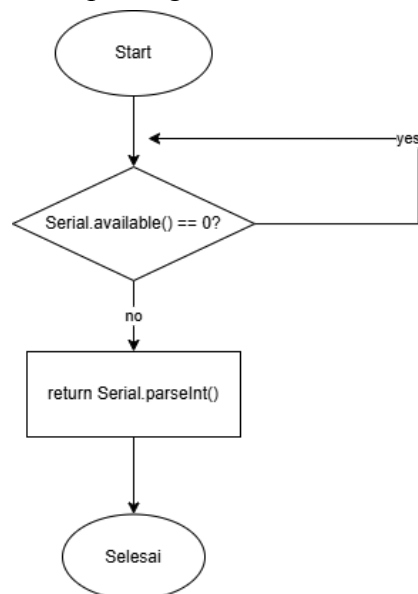
3.2.36. Flowchart Fungsi showAllItem



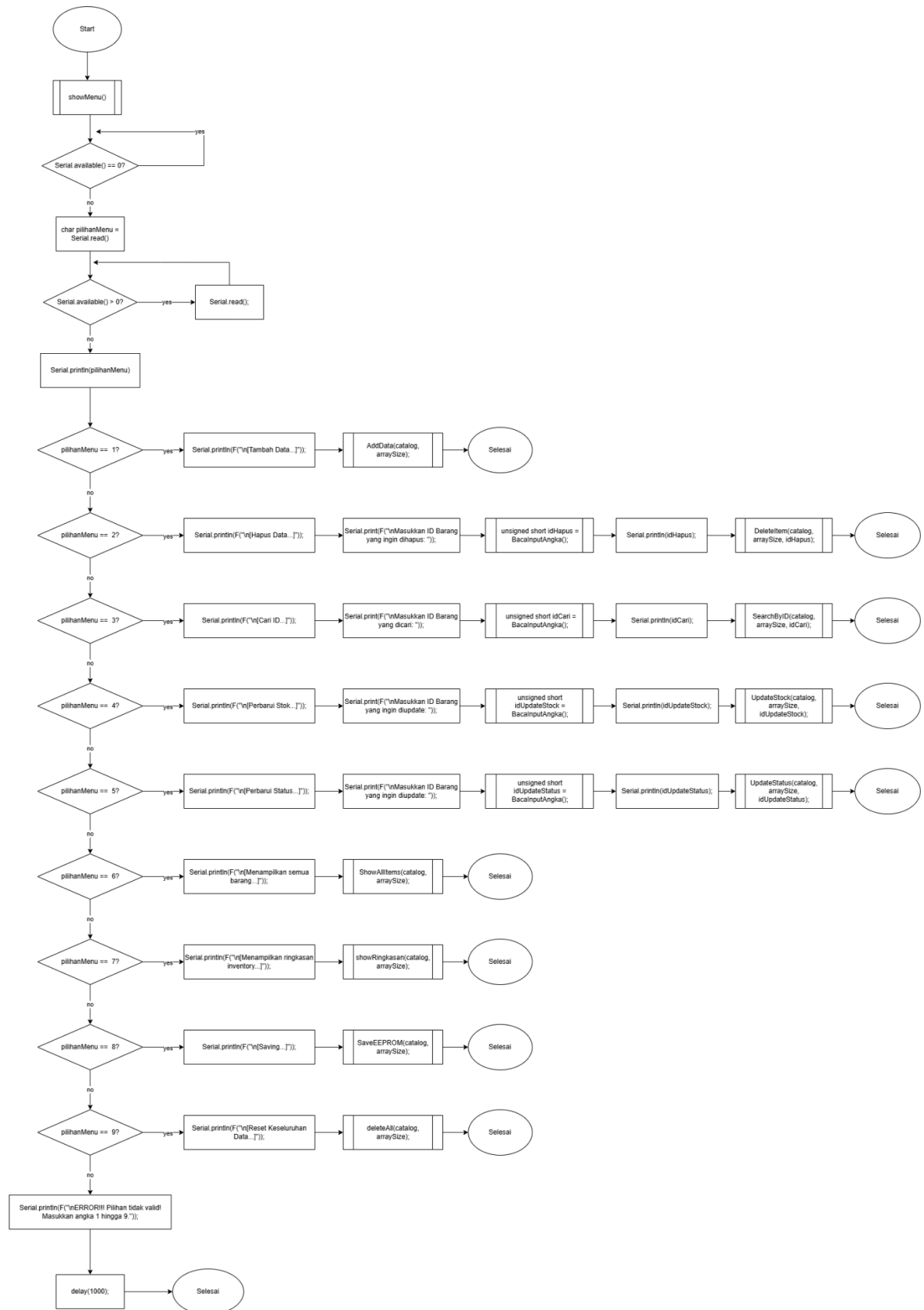
3.2.37. Flowchart Fungsi showMenu



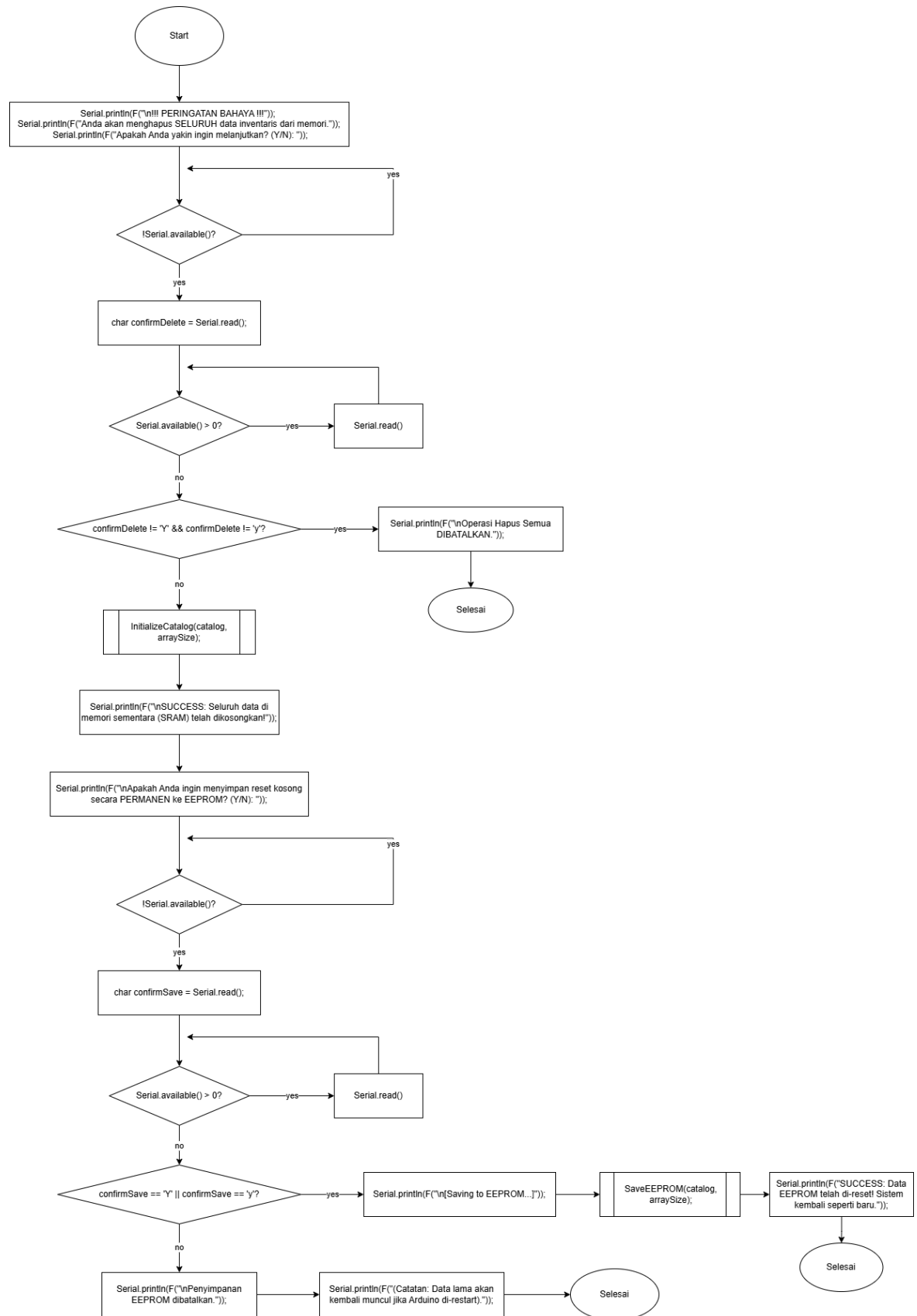
3.2.38. Flowchart Fungsi BacaInputAngka



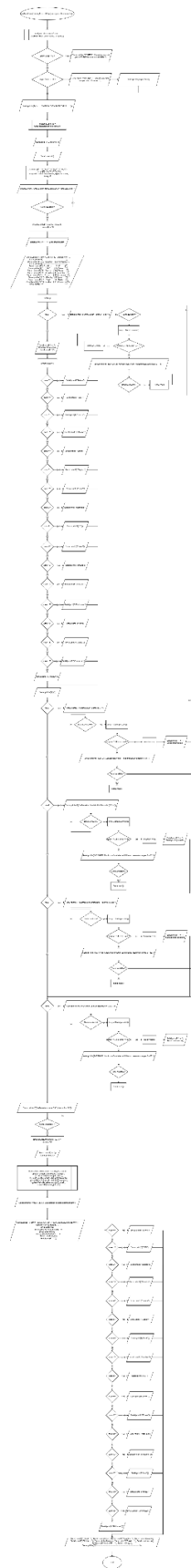
3.2.39. Flowchart Fungsi runSystem



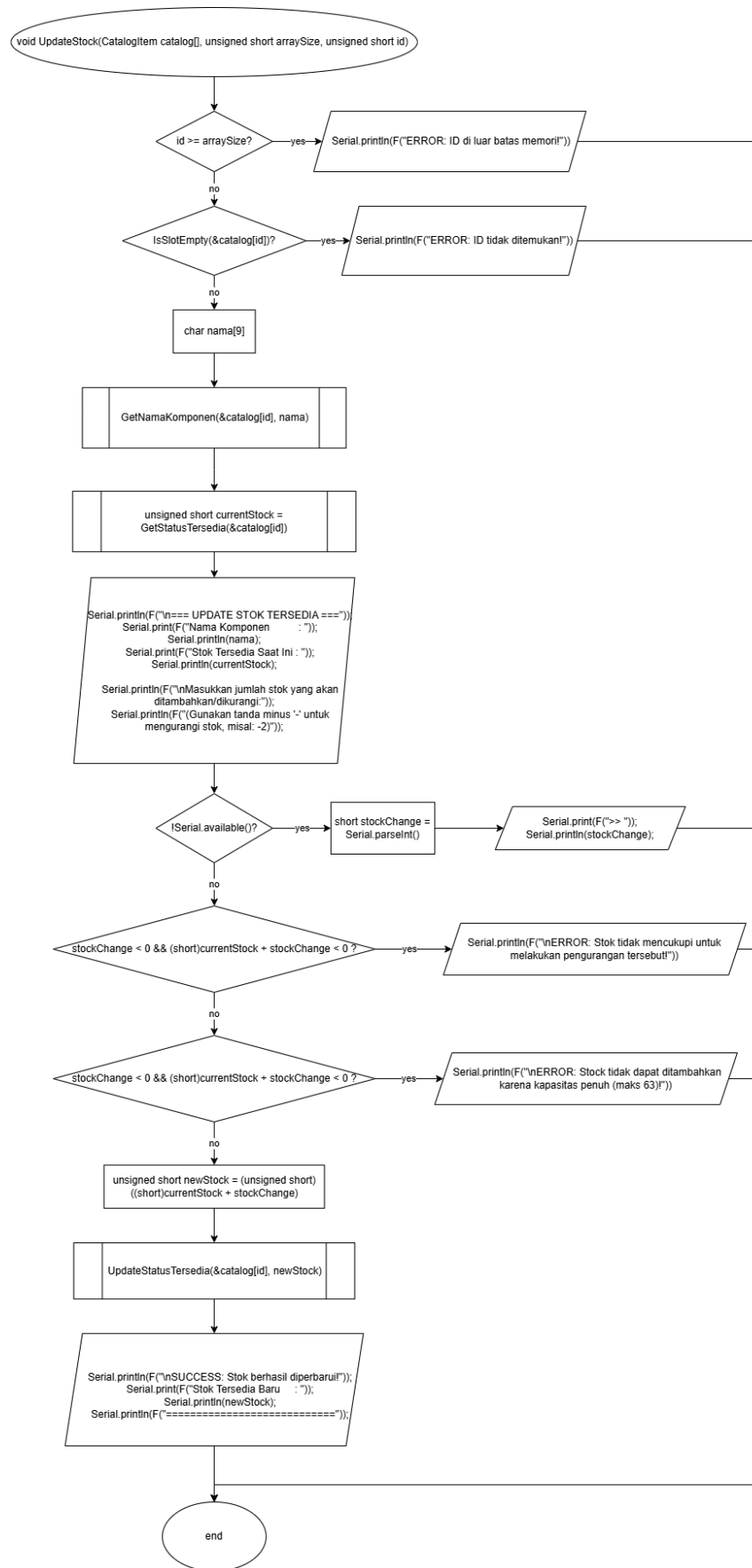
3.2.40. Flowchart Fungsi deleteAll



3.2.41. Flowchart Fungsi addData



3.2.42. Flowchart Fungsi updateStock



3.3. Penjelasan Logika dan Struktur Data

3.3.1. Penjelasan Struktur Data

Sesuai yang telah dituliskan di sub-sub-bab 2.2.1, struktur data yang digunakan adalah sebuah array dari struct seperti di tabel 2.2.1.3. Dalam implementasinya dalam kode program adalah seperti tabel 3.3.1.1 sebagai berikut :

Tabel 3.3.1.1 Rancangan Struct dalam Kode Program

```
typedef struct {  
    uint8_t nama_komponen[6];  
    uint8_t lokasi_dan_jumlah_status[3];  
    uint8_t kategori_dan_pic[3];  
} CatalogItem;
```

Struct ini terdiri dari tiga elemen yaitu nama_komponen, lokasi_dan_jumlah_status, dan kategori_dan_pic. Untuk elemen nama_komponen terdiri hanya dari variable nama komponen. Sedangkan untuk kedua elemen lainnya merupakan gabungan dari dua atau lebih variabel. Contohnya kategori_dan_pic adalah gabungan dari variabel kategori dan variabel nama PIC. Lalu untuk elemen lokasi dan jumlah_status merupakan gabungan dari variabel lokasi barang dan ketiga jumlah status barang (tersedia, dipinjam, dan rusak).

Array dari struct ini memiliki panjang sebesar 85 struct atau sebesar 1020 byte. Hal ini sesuai dengan perhitungan yang dilakukan di sub-sub-bab 2.2.1 yang berdasarkan *bottleneck* kapasitas EEPROM sebesar 1024 byte.

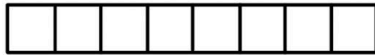
3.3.2. Penjelasan Logika CRUD dan EEPROM

Dalam membuat CRUD, dikarenakan struktur data yang telah dibuat melakukan penggabungan variabel menjadi satu elemen, maka digunakan operasi bitwise untuk mencapai hal tersebut (Pergeseran bit (<< dan >>), perkalian bit (&), dan penjumlahan bit (|)).

Legenda



: Bit



: Byte



: Lokasi

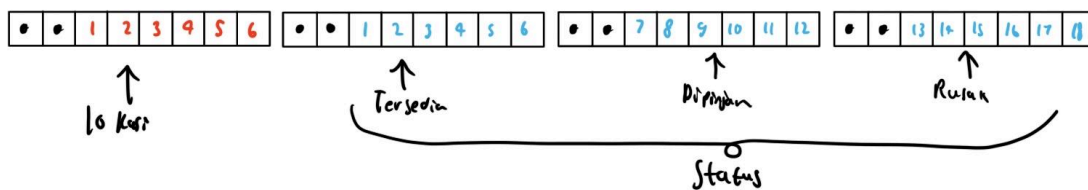


: Kosong

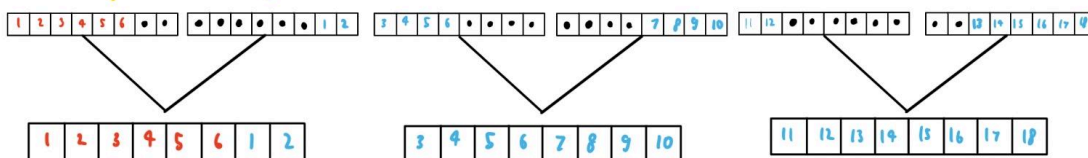


: Jumlah Status

Initial Condition



Shifting and Merger



Gambar 3.3.2.1 Ilustrasi Penggabungan Lokasi dan Jumlah Status

Lalu untuk Nama Barang dan Nama PIC harus melalui kompresi terlebih dahulu. Kompresinya dilakukan dengan mencari indeks karakter berdasarkan tabel 2.2.1.1 lalu menggunakan indeks tersebut sebagai pengganti karakter tersebut. Setelah digantikan dengan indeks, dilakukan penggabungan mirip dengan gambar 3.3.2.1.

Untuk EEPROM, akan digunakan fungsi dari library Arduino dalam melakukan manipulasi memori di EEPROM. Fungsi yang akan digunakan antara lain EEPROM.get() (Untuk membaca EEPROM) dan EEPROM.put() (Untuk menulis ulang EEPROM).

3.3.3. Penanganan Masalah Khusus

1. Pencarian ID Saat Penambahan Item

Solusi yang digunakan yaitu:

- Sistem melakukan pengecekan ID sebelum proses penambahan data untuk memastikan slot yang dituju valid dan berada dalam rentang array (0-63).
- ID digunakan sebagai indeks langsung pada array sehingga proses pencarian dilakukan konstan ($O(1)$).

- Sebelum data abru ditulis, sistem memverifikasi apakah slot pada ID tersebut masih kosong menggunakan pengecekan atribut utama.
- Jika slot sudah terisi, sistem menolak penambahan data untuk menghindari penimpaan data lama.
- Pendekatan ini memastikan integrasi data tetap terjaga pada setiap proses insert.

2. Penambahan Item Jika Catalog Sudah Penuh

Solusi yang digunakan yaitu:

- Sistem membatasi kapasitas penyimpanan maksimum sesuai ukuran array yaitu 64 slot.
- Sebelum penambahan data, sistem melakukan pengecekan jumlah slot kosong menggunakan mekanisme iterasi sebelum catalog.
- Jika tidak ditemukan slot kosong, sistem menampilkan pesan bahwa catalog telah penuh dan proses penambahan dibatalkan.
- Sistem tidak melakukan reallocasi memori karena menggunakan struktur statis berbasis array.
- Pendekatan ini menjaga stabilitas sistem pada perangkat dengan keterbatasan memori seperti Arduino UNO.

3. Status dan Stok Barang Sudah Penuh atau Kosong

Solusi yang digunakan yaitu:

- Sistem menggunakan tiga parameter status utama yaitu tersedia, dipinjam, dan rusak untuk merepresentasikan kondisi stok barang.
- Setiap perubahan stok divalidasi agar nilai tidak melebihi batas maksimum maupun menjadi nilai negatif.
- Jika nilai stok tersedia bernilai nol, sistem akan membatasi proses peminjaman atau pengurangan stok.
- Sistem juga memastikan total kombinasi status tetap konsisten setelah proses update dilakukan.
- Validasi dilakukan sebelum dan sesudah update untuk menjaga akurasi data inventaris.

Bab 4: Implementasi dan Studi Kasus

4.1. Struktur Program Modular

Dalam implementasi program ini, digunakan struktur program modular yang terdiri dari file .h (*header file*) dan file .cpp (*source file*).

4.1.1. main.cpp

Ini merupakan *source file* utama. Dalam *source file* ini terbagi menjadi dua yaitu inisialisasi saat *setup* (diawal arduino menyala) dan *main loop* yang akan loop fungsi UI terminal.

4.1.2. constant.h dan constant.cpp

Dalam file ini terdiri dari konstanta yang digunakan dalam program.

4.1.3. catalog.h dan catalog.cpp

File ini menyimpan struct katalog dan fungsi-fungsi CRUD untuk memanipulasi struct tersebut.

4.1.4. eepromControl.h dan eepromControl.cpp

File ini berisi fungsi membaca EEPROM dan memasukkannya ke SRAM dan fungsi menulis EEPROM dari data di SRAM.

4.1.5. addData.h dan addData.cpp

File ini berisi fungsi menambahkan ID item baru beserta detailnya yang diinput dari pengguna. File berisi validasi input detail agar tidak melebihi kapasitas dari Arduino.

4.1.6. delete.h dan delete.cpp

Fitur ini digunakan untuk menghapus data barang berdasarkan ID dengan cara mengosongkan slot data setelah melalui proses validasi dan konfirmasi pengguna.

4.1.7. deleteAll.h dan deleteAll.cpp

Fitur ini digunakan untuk menghapus atau mereset semua item yang ada. Ada pilihan untuk menghapus SRAM ataupun semua isi EEPROM.

4.1.8. Ringkasan.h dan Ringkasan.cpp

Fitur ini digunakan untuk meringkas isi inventory yang ada. Mulai dari total slot unique item yang tersedia, total stok masing-masing tiap status, grand total barang, dan juga notifikasi jika slot item tersedia sudah sisa 5 atau kurang.

4.1.9. SearchByID.h dan SearchByID.cpp

Fitur ini digunakan untuk mencari item berdasarkan ID. Jika ID di luar range atau isinya kosong, maka akan keluar pemberitahuan error dan fungsi kembali ke main menu. Jika ID ada, maka akan muncul detail dari item dengan ID yang bersangkutan.

4.1.10. serialUI.h dan serialUI.cpp

File ini berisi sistem utama dari program. File ini menjalankan menu dan melakukan parsing input pilihan menu dari pengguna. Input pilihan menu tersebut akan menjalankan fungsi fitur yang dipilih. Jika fitur membutuhkan nomor ID, maka program akan meminta parsing input int ID tambahan sebelum masuk ke fitur.

4.1.11. showAllItem.h dan showAllItem.cpp

Fitur ini digunakan untuk menampilkan seluruh data inventaris yang masih aktif dengan melakukan iterasi pada seluruh slot array dan melewati data yang kosong.

4.1.12. updateStatus.h dan updateStatus.cpp

Fitur ini digunakan untuk memperbarui status stok barang berdasarkan ID yang dipilih, dengan menampilkan data lama terlebih dahulu sebelum dilakukan perubahan.

4.1.13. updateStock.h dan updateStock.cpp

Fitur ini digunakan untuk memperbarui stok barang yang tersedia. Fitur melakukan validasi dan memastikan total stok tersedia tidak melebihi kapasitas Arduino.

4.2. Skenario Studi Kasus

4.2.1. Main Menu

```
#Tampilan menu
=====
          SISTEM INVENTARIS LAB EMBEDDED SYSTEMS
=====
1. Tambah Data Barang
2. Hapus Data Barang
3. Cari Data Berdasarkan ID
4. Perbarui Stok Barang
5. Perbarui Status
6. Tampilkan semua item
7. Ringkasan Inventaris
8. Save to EEPROM
9. Hapus Seluruh Data (Reset)
=====
Masukkan pilihan Anda (1-9): 6
```

4.2.2. Tambah Data Barang

```

#Tambah data secara keseluruhan
[Tambah Data...]

=== TAMBAH DATA INVENTARIS ===
ID barang: 1

Masukkan Nama Komponen (Maks 8 huruf):
>> 5K6

=== DAFTAR KATEGORI ===
-----
| ID | Kategori | ID | Kategori |
-----
| 0 | Mikon | 8 | IC |
| 1 | SBC | 9 | Driver |
| 2 | Sensor | 10 | Power |
| 3 | Aktuator | 11 | AlatUkur |
| 4 | Comm | 12 | Instrumen |
| 5 | Display | 13 | Alat |
| 6 | Pasif | 14 | Media |
| 7 | Semikon | 15 | Connector |
-----

Masukkan Kategori (0-15):
>> 6 (Pasif)

Masukkan Lokasi (0-63):
>> 12

Masukkan Jumlah Stok Tersedia (0-63):
>> 50

Masukkan Jumlah Stok Dipinjam (0-63):
>> 50

Masukkan Jumlah Stok Rusak (0-63):
>> 20

Masukkan Nama PIC (Maks 4 huruf):
>> KENN

SUCCESS: Data barang berhasil ditambahkan!

=== RINGKASAN DATA YANG DITAMBAHKAN ===
ID Barang : 1
Nama Barang : 5K6
Kategori : 6 - Pasif
Lokasi : 12
Tersedia : 50
Dipinjam : 50

```

Rusak : 20 Nama PIC : KENN =====
<i>#Jika kategori di luar range</i> Masukkan Kategori (0-15): ERROR: ID Kategori tidak valid! Harap masukkan angka 0 - 15.
<i>#Jika lokasi di luar range</i> Masukkan Lokasi (0-63): ERROR: Lokasi tidak valid! Harap masukkan angka 0 - 63.
<i>#Jika Stok di luar range</i> Masukkan Jumlah Stok Tersedia (0-63): ERROR: Stock overflow/tidak valid! Harap masukkan angka 0 - 63. Masukkan Jumlah Stok Dipinjam (0-63): ERROR: Stock overflow/tidak valid! Harap masukkan angka 0 - 63. Masukkan Jumlah Stok Rusak (0-63): ERROR: Stock overflow/tidak valid! Harap masukkan angka 0 - 63.

4.2.3. Hapus Data Barang

<i># Hapus item</i> [Hapus Data...] Masukkan ID Barang yang ingin dihapus: <u>2</u> === KONFIRMASI DELETE === ID : 2 Nama Komponen: 5K6 Yakin ingin menghapus? (Y/N): >> <u>Y</u> DATA BERHASIL DIHAPUS (nama komponen direset)
<i># Hapus item jika dibatalkan</i> [Hapus Data...] Masukkan ID Barang yang ingin dihapus: <u>2</u> === KONFIRMASI DELETE === ID : 2 Nama Komponen: 5K6

Yakin ingin menghapus? (Y/N) :

>> N

INFO: DELETE DIBATALKAN

Hapus item jika slot kosong

[Hapus Data...]

Masukkan ID barang: 5

ERROR: Data kosong, tidak bisa dihapus!

Hapus item jika di luar batas array

[Hapus Data...]

Masukkan ID barang: 99

ERROR: ID di luar batas array

4.2.4. Cari Data Berdasarkan ID

Fungsi Search by ID

Masukkan ID Barang yang dicari: 1

===== DETAIL BARANG =====

ID Barang : 1

Nama : 5K6

Kategori : 6

Lokasi : 12

PIC : KENN

--- STATUS STOCK ---

Tersedia : 50 unit

Dipinjam : 50 unit

Rusak : 20 unit

Total Stok: 120 unit

=====

Jika di luar range

[Cari ID...]

Masukkan ID Barang yang dicari: 90

ERROR!! ID 90 melebihi batas memori.

Jika isi ID kosong (tidak ada item)

[Cari ID...]

Masukkan ID Barang yang dicari: 3

ERROR!! Data dengan ID 3 tidak ditemukan.

4.2.5. Perbarui Stock Barang

```
# Perbarui stok item tersedia
[Perbarui Stok...]

Masukkan ID Barang yang ingin diupdate: 1

=== UPDATE STOK TERSEDIA ===
Nama Komponen          : 5K6
Stok Tersedia Saat Ini : 50

Masukkan jumlah stok yang akan
ditambahkan/dikurangi:
(Gunakan tanda minus '-' untuk mengurangi stok,
misal: -2)
>> -30

SUCCESS: Stok berhasil diperbarui!
Stok Tersedia Baru      : 20
=====
```

```
# Jika stok melebihi batas
[Perbarui Stok...]

Masukkan ID Barang yang ingin diupdate: 1

=== UPDATE STOK TERSEDIA ===
Nama Komponen          : 5K6
Stok Tersedia Saat Ini : 50

Masukkan jumlah stok yang akan
ditambahkan/dikurangi:
(Gunakan tanda minus '-' untuk mengurangi stok,
misal: -2)
>> 20

ERROR: Stock tidak dapat ditambahkan karena
kapasitas penuh (maks 63)!
```

4.2.6. Perbarui Status

```
# Perbarui status item
[Perbarui status...]
Masukkan ID Barang yang ingin diupdate: 1

=== DATA LAMA ===
Nama      : 5K6
Tersedia: 10
```

```
Dipinjam: 2
Rusak    : 1
```

```
Yakin ingin update status? (Y/N):
>> Y
```

```
Masukkan status baru:
Tersedia:
>> 15
```

```
Dipinjam:
>> 1
```

```
Rusak:
>> 0
```

```
SUCCESS: DATA BERHASIL DIUPDATE
```

```
# Perbarui status item ketika dibatalkan
[Perbarui status...]
Masukkan ID Barang yang ingin diupdate: 1
```

```
=== DATA LAMA ===
Nama      : 5K6
Tersedia: 10
Dipinjam: 2
Rusak     : 1
```

```
Yakin ingin update status? (Y/N):
>> N
```

```
INFO: UPDATE DIBATALKAN
```

```
#Perbarui status item ketika data kosong
[Perbarui status...]
Masukkan ID Barang yang ingin diupdate: 5
```

```
ERROR: Data kosong
```

```
#Perbarui status item ketika input diluar batas
[Perbarui status...]
Masukkan ID Barang yang ingin diupdate: 99
```

```
ERROR: ID di luar batas
```

4.2.7. Tampilkan Semua Item

```
# Tampilkan semua item
[Menampilkan semua barang...]
```

```
===== DAFTAR INVENTARIS =====
```

```
-----  
ID: 0  
Nama: 5K6  
Kategori: 6  
Lokasi: 2  
PIC: VAN  
Tersedia: 15  
Dipinjam: 2  
Rusak: 1  
-----
```

```
ID: 1  
Nama: 1K2  
Kategori: 6  
Lokasi: 40  
PIC: KEN  
Tersedia: 10  
Dipinjam: 15  
Rusak: 1  
===== SELESAI =====
```

```
#Tampilkan semua item ketika semua slot kosong  
[Menampilkan semua barang...]  
Daftar Inventaris Masih Kosong!!!
```

4.2.8. Ringkasan Inventaris

```
# Fungsi Ringkasan  
[Menampilkan ringkasan inventory...]
```

```
=====
```

RINGKASAN INVENTARIS	
----- STATUS KAPASITAS DATABASE -----	
- Slot Terpakai	: 2 / 85
- Slot Tersisa	: 83
----- STATUS KONDISI BARANG LAB -----	
- Siap Digunakan	: 70 unit
- Sedang Dipinjam	: 70 unit
- Kondisi Rusak	: 60 unit

GRAND TOTAL : 200 unit	
=====	

4.2.9. Save to EEPROM


```
# Fungsi save ke EEPROM  
[Saving...]
```

4.2.10. Hapus Seluruh Data (Reset)

```
# Fungsi reset  
[Reset Keseluruhan Data...]  
  
!!! PERINGATAN BAHAYA !!!  
Anda akan menghapus SELURUH data inventaris dari  
memori.  
Apakah Anda yakin ingin melanjutkan? (Y/N): Y  
  
SUCCESS: Seluruh data di memori sementara (SRAM) telah  
dikosongkan!  
  
Apakah Anda ingin menyimpan reset kosong secara  
PERMANEN ke EEPROM? (Y/N): Y  
  
[Saving to EEPROM...]  
SUCCESS: Data EEPROM telah di-reset! Sistem kembali  
seperti baru.
```

```
# Jika tidak jadi  
[Reset Keseluruhan Data...]  
  
!!! PERINGATAN BAHAYA !!!  
Anda akan menghapus SELURUH data inventaris dari  
memori.  
Apakah Anda yakin ingin melanjutkan? (Y/N): N  
  
Operasi Hapus Semua DIBATALKAN.
```

```
# Jika hanya reset SRAM  
[Reset Keseluruhan Data...]  
  
!!! PERINGATAN BAHAYA !!!  
Anda akan menghapus SELURUH data inventaris dari  
memori.  
Apakah Anda yakin ingin melanjutkan? (Y/N): Y  
  
SUCCESS: Seluruh data di memori sementara (SRAM) telah  
dikosongkan!  
  
Apakah Anda ingin menyimpan reset kosong secara  
PERMANEN ke EEPROM? (Y/N): N  
  
Penyimpanan EEPROM dibatalkan.  
(Catatan: Data lama akan kembali muncul jika Arduino  
di-restart).
```

Bab 5: Analisis Hasil dan Evaluasi

5.1. Analisis Implementasi

Program inventarisasi tugas besar ini memiliki penggunaan RAM sebesar 85.1% dari total 2048 byte RAM yang dimiliki Arduino UNO dan penggunaan *Flash Memory* sebesar 48.6% dari kapasitas yang dimiliki Arduino UNO. Dikarenakan program ini tidak menggunakan heap, maka penggunaan RAM tidak akan bertambah selama runtime. Sedangkan untuk penggunaan EEPROM berdasarkan perhitungan adalah sebesar 100% yaitu 1024 byte.

```
RAM:  [===== ] 85.1% (used 1742 bytes from 2048 bytes)
Flash: [===== ] 48.6% (used 15674 bytes from 32256 bytes)
```

Gambar 5.1.1 Analisis Penggunaan Memori Program

Dikarenakan program ini menggunakan array dan menggunakan indeksnya sebagai ID, dalam mengakses suatu data berdasarkan ID akan berjalan dalam kecepatan konstan atau $O(1)$. Namun, dalam menambahkan item di slot baru dikarenakan harus mencari ID yang kosong secara sekuensial, maka program akan berjalan mencari dengan kecepatan terburuk $O(N)$ dengan N : banyak barang yang sudah tersimpan.

Walaupun begitu, hal ini masih lebih baik dibanding jika menggunakan struktur data linked list. Hal ini dikarenakan linked list membutuhkan lebih banyak tempat per node dikarenakan harus menyimpan *pointer* untuk node selanjutnya (dan mungkin sebelumnya) serta dalam mengakses suatu data menggunakan ID harus dilakukan pencarian secara sekuensial sehingga kecepatan pencariannya $O(N)$. Lalu, kelebihan dalam menggunakan array dibanding linked list adalah array tidak harus memikirkan fragmentasi heap yang disebabkan oleh jika memanggil heap dan mendelete heap.

5.2. Analisis Keterbatasan Sistem

Dikarenakan program ini dijalankan pada Arduino UNO R3, hal yang menjadi *bottleneck* program ini adalah kapasitas EEPROM-nya. Hal ini menyebabkan banyak item dengan ID unik yang bisa disimpan hanya sebesar 85 item. Selain itu, dikarenakan Arduino tidak memiliki *built-in wireless communication*, Arduino harus dicolokan langsung ke device.

Bab 6: Pembagian Tugas Anggota Kelompok

NIM	NAMA	PEMBAGIAN TUGAS
13224011	Vanadia Valianti	Pembuatan fungsi delete item, fungsi update status data, fungsi show all item. Pembuatan laporan Bab 2 (2.1.2. dan 2.2.2.), Bab 3 (3.2.34., 3.2.35., 3.2.36., dan 3.3.3.), Bab 4 (4.1.6., 4.1.11., 4.1.12., 4.2.3., 4.2.6., dan 4.2.7.)
13224032	Kenneth Harrison Siswanto	Pembuatan fungsi SearchByID, fungsi Ringkasan, fungsi deleteAll, dan Serial UI untuk integrasi semua fungsi/fitur. Bab 2 (2.1.2, 2.2.2) Bab 3 (3.1.1, 3.1.2, 3.2.32, 3.2.33, 3.2.37, 3.2.38, 3.2.39, 3.2.40) Bab 4 laporan (4.1.5, 4.1.7, 4.1.8, 4.1.9, 4.1.10, 4.1.11, 4.2.2, 4.2.4, 4.2.8, 4.2.10) (PPT Bab 2
13224038	Beatrice Corryn Pangestu Siswanto	Pembuatan fungsi addData, updateStock, Bab 2 (2.1.2 dan 2.2.2), Bab 3 (3.2.41 dan 3.2.42), Bab 4 (4.2.2 dan 4.2.5), PPT bab 3,4,5.
13224071	Muhammad Aqeel Ghani	Pembuatan environment, fungsi CRUD, dan fungsi EEPROM. Pembuatan laporan Bab 1, Bab 2 (2.1.1. dan 2.2.1), Bab 3 (3.2 (Fungsi CRUD dan EEPROM), 3.3.1., dan 3.3.2.), Bab 4 (4.1.1., 4.1.2, 4.1.3, 4.1.4), Bab 5, dan Bab 7.

Bab 7: Penutup

7.1. Kesimpulan

- Sistem inventarisasi laboratorium berhasil diimplementasikan dalam Arduino UNO menggunakan struktur data Array.
- Sistem inventarisasi dapat menyimpan 85 item unik pada memori non-volatile EEPROM ATmega328P.
- Sistem inventaris memiliki struct dengan kemampuan untuk menyimpan :
 - ID Barang (indeks dari struct tersebut)
 - Nama Barang (dapat terdiri dari 8 karakter alfabet kapital, non-kapital, dan angka)
 - Lokasi (merupakan nomor loker dari 0-63)
 - Jumlah status barang (Tiap status dapat terisi 0-63)
 - Kategori (Terdapat 16 kategori dengan indeks kategori 0-15)
 - Nama PIC (dapat terdiri dari 8 karakter alfabet tidak case sensitive)
- Sistem inventarisasi ini memiliki beberapa fungsi seperti UI Main Menu, Tambah Data Barang, Hapus Data Barang, Cari Data Berdasarkan ID, Perbarui Stock Barang, Perbarui Status, Tampilkan Semua Item, Ringkasan Inventaris, Save to EEPROM, dan Hapus Seluruh Data (Reset).
- Sistem inventarisasi memiliki penggunaan RAM sebesar 85.1% dari total 2048 byte RAM yang dimiliki Arduino UNO dan penggunaan Flash Memory sebesar 48.6% dari kapasitas Flash yang dimiliki Arduino UNO.

7.2. Saran Pengembangan

- Menggunakan penyimpanan eksternal supaya jumlah data yang dapat disimpan tidak ter-*bottleneck* oleh kapasitas EEPROM (Namun, kemungkinan besar akan ter-*bottleneck* oleh kapasitas SRAM).
- Implementasi GUI sehingga *user experience* dapat lebih mudah (Namun, berarti di device yang terkoneksi dengan Arduino harus memiliki program custom)
- Implementasi penggunaan QR code atau barcode untuk mempermudah mencari item (namun berarti harus menggunakan modul eksternal atau aplikasi eksternal dikarenakan ini merupakan proses yang cukup berat)
- Sistem inventaris dapat diakses secara *wireless* (namun harus menggunakan modul tambahan wifi/bluetooth atau menggunakan microcontroller yang memiliki kapabilitas tersebut)

Daftar Pustaka

- [1] Arduino, “UNO R3 | Arduino Documentation,” *docs.arduino.cc*, 2025. <https://docs.arduino.cc/hardware/uno-rev3/>
- [2] Arduino, “Arduino Memory Guide | Arduino Documentation,” *docs.arduino.cc*, 2023. <https://docs.arduino.cc/learn/programming/memory-guide>

Lampiran

Source Code :

<https://github.com/AqeelGhani/tugas-besar-pmp-2026-kelompok-6>



Docs Laporan (Jika ingin melihat version history untuk memvalidasi pembagian tugas laporan, dapat mengontak Muhammad Aqeel Ghani (NIM : 13224071) untuk diberikan akses):

https://docs.google.com/document/d/1oRgPSkJoC6zzD_uECA_AX2t_n-t5MROtjOMcz_kANm0/edit?usp=sharing



Video Presentasi dan Demo :

https://youtube.com/playlist?list=PLRs_edC3ZZ_gmnUyWeE1VLXdyuMIJ6Fj5&si=M54ECsvDxnEOm5Ek

